



फाइनली वी आर स्टार्टिंग जनरे...
TXT 570.78KB

make notes for me out of this transcript , explain everything properly deeply

Generative AI Series — Complete Notes (Video 1: LLM Foundation)

Introduction

Instructor: Aakash Vyas | **Channel:** Sheryians AI School

This is the **first video** of the Generative AI series. The full series is divided into **4 parts**:

Video	Topic
1	LLM Foundation (this video)
2	RAG Systems (Retrieval Augmented Generation)
3	Agentic AI
4	Big Project with Deployment

Small projects are built throughout all videos.

Prerequisites Before Starting GenAI

There are **5 things** you should know before diving into Generative AI:

1. A Programming Language (Python recommended)

- Python is used throughout this channel.
- If you know JavaScript, you can check the "Sheryians Coding School" channel.

- We are **implementing GenAI using Python** — so Python is required.

2. Machine Learning Fundamentals

- Honestly, you **don't strictly need ML/DL** to use LLM APIs.
- But for interviews (GenAI Engineer / AI Engineer roles), interviewers may ask about algorithms and architectures.
- Basic knowledge is important.

3. Deep Learning Concepts

- Same as above — basics needed for interviews.

4. Transformers Architecture

- **GPT = Generative Pre-trained Transformers**
- The entire GenAI is based on Transformer architecture.
- Watch the 4th video of the Deep Learning series for this.

5. Basic NLP (Natural Language Processing)

- NLP = Natural Language Processing (processing Hindi, English, etc.)
- Concepts needed: **Tokenization, Embeddings**, etc.
- Watch the NLP video for this.



Evolution of Generative AI — The 4 Phases

Phase 1: Research

- Everything starts with **research**.
- Example research papers:
 - **LSTM** (Long Short-Term Memory)
 - **GRU** (Gated Recurrent Units)
 - **"Attention Is All You Need"** (Google Brain) — introduced Transformers

- Research papers **prove** what's possible with data and models.

Phase 2: Base/Foundation Models (LLMs)

- Big companies (OpenAI, Google, Anthropic, xAI, Amazon) take research papers and **create base models**.
- Examples:
 - OpenAI → **ChatGPT (GPT models)**
 - Anthropic → **Claude**
 - Google → **Gemini**
 - Meta → **Llama**
 - xAI (Elon Musk) → **Grok**
 - Mistral → **Mistral models**
 - DeepSeek → **DeepSeek (open source)**

Phase 3: Providers

- These companies become **providers** — they give you **API keys** to use their models.
- You can use their:
 - API pricing
 - API platform
 - Agents
- Examples:
 - OpenAI Platform (GPT-5.2, GPT-5.2 Pro, GPT-5 Mini)
 - Google AI Studio (Gemini)
 - Grok API
 - Mistral AI Studio
 - **Hugging Face** — free models & datasets

Phase 4: You (The Developer)

- **You** use these LLMs to create great AI applications.
- After learning GenAI, you can call yourself a **GenAI Engineer** (after practice).

What is an LLM?

LLM = Large Language Model

Breaking it down:

Large

- Trained on **huge amounts of text data**: books, websites, Wikipedia, articles, code, conversations, etc.
- Training requires **huge GPU power**.
- Big companies spend money to buy land, set up GPUs, and train models.
- A normal person cannot train these on a laptop.

Language

- Understands and generates **human language**: English, Hindi, Spanish, even programming languages like Python.
- ChatGPT can translate English ↔ Hindi easily because it was trained on **multiple languages**.

Model

- A **Deep Learning system** (Neural Network) that learns patterns from data.
- Built using **Transformer architecture**.
- A "model" is anything you create using ML/DL.

How LLMs Actually Work (Important!)

- LLMs **do NOT** search Google.
- They **do NOT** think like humans.
- They **do NOT** understand meaning.
- They **convert everything into numbers** → called **Embeddings**.
- They **look at patterns** and **predict the next most likely word**.
- They keep predicting word-by-word until the answer is complete.

Example: "What is Python?" → LLM doesn't search Google. It converts the sentence to embeddings (numbers like 0,1,0,0,1...), finds similar embeddings, and predicts the next word repeatedly.

This conversion to numbers is called **Natural Language Processing (NLP)**.

🌟 Popular LLM Models

Model	Company	Notes
ChatGPT (GPT)	OpenAI	Various types (5.2, 5.2 Pro, 5 Mini, 4.1, 4.1 Mini, 4.1 Nano)
Gemini	Google	Fast, Pro, multiple types
Llama	Meta	Open models
Claude	Anthropic	Various types
Grok	xAI (Elon Musk)	Multiple models
Mistral	Mistral AI	Some free models (e.g., Mistral Small)
DeepSeek	DeepSeek	Open source, free on Hugging Face

- **Paid models:** GPT, Gemini (mostly), Claude, Grok
- **Free models:** Mistral (some), DeepSeek, Hugging Face models

We **don't build these from scratch** — we learn to **use pre-built LLMs efficiently**.
That's what Generative AI is.

🔄 The Problem: Different SDKs for Different Providers

If every provider has a different SDK:

- Different syntax
- Different response format
- You'd have to learn 100+ different codes

Solution: LangChain

What is LangChain?

- An **open-source framework** with pre-built agent architecture.
- Connects to OpenAI, Anthropic, Google, and more.
- **Same amount of code** works for different models.
- You just need **API keys** for each provider.

"LangChain changed everything."

LangChain's 6 Main Components

Component	Purpose
1. Models	The LLMs themselves (chat, embedding, multimodal)
2. Prompts	Instructions given to the model
3. Chains	Sequence of steps connected together
4. Memory	Remember past interactions
5. Indexes	Connect external data (PDFs, databases) — RAG
6. Agents	AI systems that decide actions dynamically using tools

Types of Models (Practical GenAI)

There are **3 main types** of models used in practical GenAI:

1. Chat Models (Language Models)

- **Job:** Generate text, answer questions, write quizzes, summarize content.
- **Example:** ChatGPT, Gemini, Claude.

2. Embedding Models

- **Job:** Convert text into numbers (vectors).
- Used in **search and RAG**.
- Help **find similar content**.
- Example: Convert a PDF into embeddings to feed into an LLM.

3. Multimodal Models

- Work with **more than just text**: images, audio, video, different file types.
- Examples: Image-text-to-text, image-text-to-image, visual question answering.

All of these are available on **Hugging Face**.

Prompts — The Instructions

Prompt = The instruction we give to a language model.

- Without prompts, the model has **no direction**.
- Better prompt → Better output.
- LLMs are **not mind readers** — quality depends on how clearly we instruct.

4 Types of Prompts

1. Simple Prompt (Direct Instruction)

- **Example:** "Explain what is Machine Learning. Basic, straightforward."
- Just a direct question.

2. System + User Prompt (Roles)

- Modern chat models use **roles**:
 - **System** → Sets behavior
 - **User** → Asks questions
- **Example:**
 - System: "You are a helpful AI teacher."
 - User: "Explain LLM in simple terms."

- System controls **tone and style**.

3. Prompt Templates (Reusable)

- Instead of writing prompts manually every time, create **reusable templates**.
- **Example:** "Explain [TOPIC] in simple terms."
- Replace [TOPIC] with anything — LLM, Machine Learning, etc.

4. Structured Prompts

- Used when you want **JSON output, bullet points, fixed format**.
- Response in JSON format with keys, definitions, examples.
- Makes output **machine-readable**.
- Used in **backend development**.

| All LLMs (OpenAI, Gemini, etc.) can handle these prompt types.

Chains

- A **sequence of steps** connected together.
- Instead of one prompt → one response, you create:
 - Step 1 → Step 2 → Step 3 → Final Output

Example:

User asks: "Summarize this article and translate it into Hindi."

- **Step 1:** Summarization of text
- **Step 2:** Translate the summary to Hindi
- **Step 3:** Return the result

| Chains are used when you need **multiple tasks** from one LLM.

Memory

- Allows the AI system to **remember past interactions**.

- Without memory: every message is treated independently.
- With memory: model can use **previous context**.

Example:

- User: "My name is Aakash."
- Later: "What is my name?"
- **Without memory:** "I don't know."
- **With memory:** "Your name is Aakash."

Very useful in LangChain for chatbots.

Indexes

- Allow us to **connect external data** to an LLM.
- LLM only knows what it was **trained on**.
- Indexes help connect:
 - PDFs
 - Documents
 - Databases
 - Company data

Example:

- 4 books. LLM trained on 3.
- 4th book connected via **Indexes** as external data.
- This system is called **Retrieval**.
- Leads to **RAG (Retrieval Augmented Generation)**.

LLMs don't know your company data, private documents, or real-time updates.
Indexes solve this.

Agents

- AI systems that can **decide what actions to take**.
- Use **tools** (web search, calculations, API calls, databases).
- Unlike chains (fixed steps), agents **dynamically decide** the next step.

Example:

- Normal chain: Fixed steps.
- Agent: For booking a flight ticket + planning a Goa trip:
 - One LLM for planning
 - One LLM for ticket booking
 - One LLM for hotel booking/negotiation
 - Multiple LLMs working together = **Agentic System**

Agents need **tools**. We'll learn tools when we move to Agents.



Practical Setup — Coding Environment

Prerequisites:

1. **VS Code** installed
2. **Latest Python** version installed

Steps:

1. Create a New Folder

- Name it: **Generative AI**
- Open in VS Code

2. Open Terminal

- Create a **Virtual Environment**

3. What is a Virtual Environment?

- All dependencies/libraries stay **inside the folder**.
- Doesn't mess with your base system.

- Always create a virtual environment for any project (GenAI, ML, DL).

4. Install UV (Fast Python Package Manager)

- **UV** is an extremely fast Python package & project manager written in Rust.
- **10–100x faster than pip.**
- Install via terminal (Mac/Linux: curl command; Windows: PowerShell command).

5. Create Virtual Environment

```
bash
```

```
uv venv
```

- Creates a `.venv` folder with bins, libraries, cache.

6. Activate Virtual Environment

```
bash
```

```
source .venv/bin/activate
```

- You'll see `(generative-ai)` in terminal.

7. Install LangChain

```
bash
```

```
uv pip install langchain
```

- If error: No pyproject.toml found → USE `uv pip install langchain`

8. Verify Installation

```
python
```

```
import langchain
print(langchain.__version__)
```

Getting API Keys

1. OpenAI API Key (Paid)

- Go to OpenAI API Keys page
- Sign up / Log in
- Create new secret key
- Copy and save in `.env` file as:

text

```
OPENAI_API_KEY=your_key_here
```

- **Cost:** ~\$5 recommended (don't spend more)
- **Pricing:** Different for different models

2. Grok API Key (Free/Paid)

- Go to Grok Cloud
- Log in → Create API key
- Save as:

text

```
GROK_API_KEY=your_key_here
```

3. Gemini API Key (Free tier available)

- Go to **Google AI Studio**
- Log in → Get API key
- Save as:

text

```
GOOGLE_API_KEY=your_key_here
```

- Free tier available (limited tokens)

4. Mistral API Key (Free tier)

- Go to **Mistral AI Studio**
- Choose **Experiment for Free** plan
- Verify phone number
- Create API key
- Save as:

text

MISTRAL_API_KEY=your_key_here

5. Hugging Face Access Token (Free)

- Go to Hugging Face → Profile → Access Tokens
- Create new token (Read access)
- Save as:

text

HUGGINGFACEHUB_ACCESS_TOKEN=your_token_here

Important Security Note:

- **Never share your API keys** with anyone or any AI.
- Store them in `.env` file only.
- Add `.env` to `.gitignore` if using Git.

Requirements File

Create `requirements.txt` with:

text

```
langchain
langchain-core
langchain-community
langchain-openai
langchain-google-genai
```

```
langchain-grok
langchain-mistralai
langchain-huggingface
python-dotenv
streamlit
sentence-transformers
transformers
torch
torchvision
torchaudio
```

Install with:

```
bash
```

```
uv pip install -r requirements.txt
```

Tip: Don't lock versions — let students download the latest. If something breaks, use ChatGPT to debug.

Using LLMs with LangChain

Step 1: Load Environment Variables

```
python

from dotenv import load_dotenv
load_dotenv()
```

Step 2: Initialize Chat Model

Option A: OpenAI (Using `init_chat_model`)

```
python

from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-4.1") # or "gpt-5", "gpt-4.1-nano"
response = model.invoke("What is cricket?")
print(response.content)
```

Option B: OpenAI (Using `ChatOpenAI`)

python

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4.1")
response = model.invoke("What is cricket?")
print(response.content)
```

Option C: Google Gemini (Using `init_chat_model`)

python

```
from langchain.chat_models import init_chat_model

model = init_chat_model("gemini-2.5-flash-lite") # Google model
response = model.invoke("Give me a paragraph on Machine Learning.")
print(response.content)
```

Option D: Google Gemini (Using `ChatGoogleGenerativeAI`)

python

```
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash-lite")
response = model.invoke("Give me a paragraph on Machine Learning.")
print(response.content)
```

Option E: Grok (Using `init_chat_model`)

python

```
from langchain.chat_models import init_chat_model

model = init_chat_model("groq:meta-llama/llama-4-scout-17b-16e-instruct") # Grok provide
r
response = model.invoke("Give me a paragraph on Machine Learning.")
print(response.content)
```

Option F: Grok (Using `chatGroq`)

python

```
from langchain_groq import ChatGroq

model = ChatGroq(model="meta-llama/llama-4-scout-17b-16e-instruct")
response = model.invoke("Give me a paragraph on Machine Learning.")
print(response.content)
```

Option G: Mistral (Using chatMistralAI)

python

```
from langchain_mistralai import ChatMistralAI

model = ChatMistralAI(model="mistral-small-2506") # Free model
response = model.invoke("Give me a paragraph on Machine Learning.")
print(response.content)
```

Option H: Hugging Face (Using API)

python

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint

llm = HuggingFaceEndpoint(
    repo_id="deepseek-ai/DeepSeek-R1",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)
response = model.invoke("Who are you?")
print(response.content)
```

Option I: Hugging Face (Local Model)

python

```
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline

llm = HuggingFacePipeline.from_model_id(
    model_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation",
    pipeline_kwargs={"max_new_tokens": 100}
)
model = ChatHuggingFace(llm=llm)
response = model.invoke("What is Data Science?")
print(response.content)
```

Important Parameters

1. Temperature

- Value: **0 to 1**
- **0** → Deterministic, reasoning, logical tasks (math, logic)
- **0.7–0.9** → Creative tasks (poems, stories, jokes)
- **Example:** Temperature 0 → same poem every time; Temperature 0.9 → different, advanced poem.

2. Max Tokens

- Limits the number of tokens generated.
- **Example:** `max_tokens=20` → Only ~20 words generated.
- **Why?** To control cost — prevent burning too many tokens.
- **Note:** Tokens can be words or sentences depending on the model.

3. Max Concurrent Requests

- Used later in the series.

Hugging Face — Free Models & Datasets

What is Hugging Face?

- One of the **biggest AI communities**.
- Hosts **tons of models and datasets** — free to use.
- Open-source models available.

Why Open Source?

- Someone creates an LLM and shares it openly.
- Others can contribute, improve, and use it.

- Thousands of LLMs emerged during the AI boom.

Types of Models on Hugging Face:

- **Multimodal:** Audio-text-to-text, image-text-to-text, image-text-to-image, visual question answering, computer vision.
- **NLP:** Text generation, classification, etc.
- **Audio:** Speech-to-text, text-to-speech, etc.

How to Use Hugging Face Models:

Via API (Server-based):

- You use Hugging Face servers → limited free usage → then paid.

Locally (Download the model):

- Download the entire model to your system.
- **Free unlimited usage** (uses your own GPU/CPU).
- **Downside:** Large models need huge storage (DeepSeek R1 = ~700 GB, OpenAI models = TBs).

Example: Using DeepSeek R1 via API

```
python
```

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint

llm = HuggingFaceEndpoint(
    repo_id="deepseek-ai/DeepSeek-R1",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)
response = model.invoke("Who are you?")
print(response.content)
```

Example: Using TinyLlama Locally

```
python
```

```
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline

llm = HuggingFacePipeline.from_model_id(
    model_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation",
    pipeline_kwargs={
        "max_new_tokens": 100,
        "temperature": 0.7,
        "do_sample": True
    }
)

model = ChatHuggingFace(llm=llm)
response = model.invoke("What is Data Science?")
print(response.content)
```

Note: You may need to install `transformers`, `torch`, `torchvision`, `torchaudio`, and `sentence-transformers` for local models.

Embeddings — Deep Dive

What are Embeddings?

- **Vector representation** of data (text → numbers).
- Convert text into **numerical form** (e.g., 0.6, 0.2, 1.9...).
- Used to **find similar content**.

Real-World Example: Movie Recommendation System

1. You have 1000 movies' data (stored line by line).
2. You bring a new movie.
3. Create **embeddings** for all 1000 movies.
4. Create embedding for the new movie.
5. **Compare** embeddings → find closest match → recommend.

Why Embeddings?

- LLMs only understand **numbers**.

- To feed external data (PDFs, documents) into LLMs, you must convert to embeddings.
- Used in **RAG applications**.

Embedding Models:

OpenAI Embeddings (Paid)

python

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
vector = embeddings.embed_query("You are going to learn GenAI.")
print(vector)
```

- **Cost:** \$0.13 per million tokens
- **Dimensions:** Up to 3072 (can set custom, e.g., 64)

OpenAI Embeddings for Multiple Documents

python

```
texts = [
    "Hello, this is Aakash Vyas.",
    "Hello, your name is YouTube.",
    "You all are very beautiful."
]
vectors = embeddings.embed_documents(texts)
print(vectors)
```

Hugging Face Embeddings (Free)

python

```
from langchain_huggingface import HuggingFaceEmbeddings

model_name = "sentence-transformers/all-MiniLM-L6-v2"
embeddings = HuggingFaceEmbeddings(model_name=model_name)

texts = [
    "Hello, this is Aakash Vyas.",
    "Hello, your name is YouTube.",
    "You all are very beautiful."
]
```

```
vectors = embeddings.embed_documents(texts)
print(vectors)
```

- **Dimensions:** 384-dimensional dense vector
- **Free** — runs locally
- May need `sentence-transformers` installed

Building a Chatbot (First Project)

Basic Chatbot (No Memory)

```
python

from dotenv import load_dotenv
load_dotenv()

from langchain_mistralai import ChatMistralAI

model = ChatMistralAI(model="mistral-small-2506")

while True:
    prompt = input("You: ")
    if prompt.lower() == "0":
        break
    response = model.invoke(prompt)
    print("Bot:", response.content)
```

Problem: No memory — bot forgets previous conversation.

Chatbot with Memory (Message History)

```
python

from dotenv import load_dotenv
load_dotenv()

from langchain_mistralai import ChatMistralAI
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage

model = ChatMistralAI(model="mistral-small-2506")
```

```
messages = [
    SystemMessage(content="You are a funny AI agent. Reply to every message in a funny way.")
]

print("Welcome! Type 0 to exit the application.")
print("-" * 50)

while True:
    prompt = input("You: ")
    if prompt == "0":
        break

    messages.append(HumanMessage(content=prompt))
    response = model.invoke(messages)
    messages.append(AIMessage(content=response.content))
    print("Bot:", response.content)

print("\nChat History:")
for msg in messages:
    print(f"{msg.__class__.__name__}: {msg.content}")
```

Problems with Simple Memory:

1. **No role separation** — can't tell who said what (system/user/AI).
2. **Raw strings** — no conversation structure.
3. **Memory grows infinitely** — will hit token limits.
4. **API cost increases** over time.
5. **Slower responses** as memory grows.
6. **No trimming mechanism** — no summarization of old chats.
7. **Not production scalable.**

Solution: Role Separation with Messages

Role	Purpose
System Message	Defines how AI should behave (tone, personality, rules)
Human Message	Represents user input

Role	Purpose
AI Message	Represents model's response

```
python
```

```
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage

messages = [
    SystemMessage(content="You are a sad AI agent. Reply to every message in a sad way."),
    HumanMessage(content="Hello"),
    AIMessage(content="Hello... I'm here but I wish I wasn't."),
    HumanMessage(content="What is linear regression?"),
]

response = model.invoke(messages)
print(response.content)
```

Dynamic Personality Selection (Funny/Sad/Angry)

```
python
```

```
print("Choose your AI mode:")
print("Press 1 for Angry mode")
print("Press 2 for Funny mode")
print("Press 3 for Sad mode")

choice = int(input("Tell your response: "))

if choice == 1:
    mode = "You are a very angry AI agent. Respond aggressively and impatiently."
elif choice == 2:
    mode = "You are a funny AI agent. Add jokes and humor to every response."
elif choice == 3:
    mode = "You are a sad AI agent. Respond in a sad and melancholic way."
else:
    mode = "You are a helpful AI assistant."

messages = [
    SystemMessage(content=mode)
]

while True:
    prompt = input("You: ")
```

```

if prompt == "0":
    break
messages.append(HumanMessage(content=prompt))
response = model.invoke(messages)
messages.append(AIMessage(content=response.content))
print("Bot:", response.content)

```

Adding Streamlit UI

python

```

import streamlit as st
from dotenv import load_dotenv
load_dotenv()

from langchain_mistralai import ChatMistralAI
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage

st.title("AI Chatbot")

model = ChatMistralAI(model="mistral-small-2506")

# Session state for chat history
if "messages" not in st.session_state:
    st.session_state.messages = []

# Sidebar for mode selection
mode = st.sidebar.selectbox(
    "Choose your AI mode",
    ["Funny", "Sad", "Angry"]
)

if mode == "Funny":
    system_content = "You are a funny AI agent. Add jokes and humor."
elif mode == "Sad":
    system_content = "You are a sad AI agent. Respond in a sad way."
else:
    system_content = "You are an angry AI agent. Respond aggressively."

# Initialize with system message
if not st.session_state.messages:
    st.session_state.messages.append(SystemMessage(content=system_content))

# Display chat history
for msg in st.session_state.messages:
    if isinstance(msg, HumanMessage):

```

```

        st.chat_message("user").write(msg.content)
    elif isinstance(msg, AIMessage):
        st.chat_message("assistant").write(msg.content)

# Chat input
if prompt := st.chat_input("Type your message..."):
    st.session_state.messages.append(HumanMessage(content=prompt))
    st.chat_message("user").write(prompt)

    response = model.invoke(st.session_state.messages)
    st.session_state.messages.append(AIMessage(content=response.content))
    st.chat_message("assistant").write(response.content)

```



Prompt Templates

What are Prompt Templates?

- Reusable prompts with placeholders.
- Instead of writing the same prompt every time, create a template.
- Fill in the blanks with variables.

Example:

python

```

from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "Please summarize the following paragraph: {paragraph}"
)

final_prompt = template.invoke({"paragraph": "Your long paragraph here..."})
print(final_prompt)

```

Chat Prompt Templates (with Roles)

python

```

from langchain_core.prompts import ChatPromptTemplate

```

```

chat_template = ChatPromptTemplate.from_messages([
    ("system", "You are a professional movie information extraction assistant. "
        "Your task is to extract useful structured information from a movie paragraph "
        "and present it in a clean readable format. "
        "Rules: Do not add any explanation. Do not add extra commentary. "
        "Follow the exact format. If information is missing, write null. "
        "Keep summary short. Do not guess unknown facts. "
        "Output format: Movie Title, Release Year, Genre, Director, Main Cast, "
        "Setting/Location, Plot, Themes, Rating, and a Short Summary."),
    ("human", "Extract information from this paragraph: {paragraph}")
])

final_prompt = chat_template.invoke({"paragraph": "Your movie paragraph here..."})

```

Using the Template with a Model

```

python

from langchain_mistralai import ChatMistralAI

model = ChatMistralAI(model="mistral-small-2506")

# Get user input
paragraph = input("Give your paragraph: ")

# Create final prompt
final_prompt = chat_template.invoke({"paragraph": paragraph})

# Invoke model
response = model.invoke(final_prompt)
print(response.content)

```

Why Use Prompt Templates?

- **Reusability** — Write once, use many times.
- **Consistency** — Same format every time.
- **Efficiency** — No need to rewrite prompts.
- **Real-world use case:** Movie information extraction company (Sinés Analytics).



Structured Output (JSON)

What is Structured Output?

- AI responses follow a **fixed, machine-readable format** (like JSON).
- Instead of natural language, you get **structured data**.
- Used in **backend systems**, databases, APIs.

Why Structured Output?

- Companies need **data**, not text.
- Can **store, search, filter, recommend, analyze**.
- Can be sent to **APIs**.
- Example: Store movie data in MongoDB/SQL.

Human-Friendly vs Machine-Friendly:

Human-Friendly:

"The movie Interstellar was released in 2014 and directed by Christopher Nolan."

Machine-Friendly (JSON):

```
json

{
  "title": "Interstellar",
  "year": 2014,
  "director": "Christopher Nolan"
}
```

Schema — The "Form" for AI

- A **schema** is like a school admission form.
- It tells you **what to fill** and **where**.
- You can't write your favorite cartoon in the "Age" box.
- AI must follow the schema.

Using Pydantic for Structured Output

Step 1: Import Pydantic

python

```
from pydantic import BaseModel
from typing import List, Optional
```

Step 2: Create a Schema (Class)

python

```
class Movie(BaseModel):
    title: str
    release_year: Optional[int] = None
    genre: List[str]
    director: Optional[str] = None
    cast: List[str]
    rating: Optional[float] = None
    summary: str
```

Step 3: Create a Parser

python

```
from langchain_core.output_parsers import PydanticOutputParser

parser = PydanticOutputParser(pydantic_object=Movie)
```

Step 4: Create Prompt Template with Format Instructions

python

```
from langchain_core.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate.from_messages([
    ("system", "Extract movie information from the paragraph. {format_instructions}"),
    ("human", "{paragraph}")
])

# Partial fill the format instructions
final_template = chat_template.partial(
```

```
format_instructions=parser.get_format_instructions()  
)
```

Step 5: Invoke the Model

python

```
from langchain_mistralai import ChatMistralAI  
from dotenv import load_dotenv  
load_dotenv()  
  
model = ChatMistralAI(model="mistral-small-2506")  
  
paragraph = input("Give your paragraph: ")  
final_prompt = final_template.invoke({"paragraph": paragraph})  
response = model.invoke(final_prompt)  
  
# Parse the response  
movie_data = parser.parse(response.content)  
print(movie_data)
```

Result:

json

```
{  
  "title": "3 Idiots",  
  "release_year": 2009,  
  "genre": ["Comedy", "Drama"],  
  "director": "Rajkumar Hirani",  
  "cast": ["Aamir Khan", "R. Madhavan", "Sharman Joshi", "Kareena Kapoor", "Boman Iran  
i"],  
  "rating": 8.4,  
  "summary": "Two friends search for their long-lost companion..."  
}
```

Why Pydantic? It ensures the AI's output follows the **exact schema** — like a teacher checking a student's form.



Summary of Key Concepts

Concept	What It Is	Why It Matters
LLM	Large Language Model	Generates text, answers questions
Embeddings	Text → Numbers (vectors)	Find similar content, RAG
Prompts	Instructions to LLM	Better prompt = Better output
Prompt Templates	Reusable prompts	Consistency, efficiency
Chains	Sequence of steps	Multi-task workflows
Memory	Remember past interactions	Contextual conversations
Indexes	Connect external data	RAG, PDFs, databases
Agents	Decide actions dynamically	Tools, API calls, web search
Structured Output	JSON, machine-readable	Databases, APIs, backend
Pydantic	Schema validation	Ensures correct format
LangChain	Framework for LLMs	Unified SDK for all providers
Hugging Face	Model & dataset hub	Free models, open source
Temperature	Creativity control (0–1)	0 = logical, 1 = creative
Max Tokens	Output limit	Cost control

What's Next?

Video 2: RAG Systems (Retrieval Augmented Generation)

- Connect external documents to LLMs.
- Learn **Runnables** (formerly Chains).
- Multiple things to learn.

Video 3: Agentic AI

- Build agent systems using LangGraph.
- Multiple AI agents working together.

Video 4: Big Project with Deployment

- Full project with deployment.
- Scalable models.
- Real-world application.



Key Takeaways

1. **GenAI = Using pre-built LLMs efficiently**, not building from scratch.
2. **LangChain** solves the problem of different SDKs for different providers.
3. **Prompts are everything** — better prompts = better output.
4. **Memory + Role Separation** = Better chatbots.
5. **Structured Output (JSON)** is crucial for real-world applications.
6. **Embeddings** are the bridge between text and LLMs.
7. **Hugging Face** offers free models and datasets.
8. **Temperature** controls creativity vs. logic.
9. **Always use virtual environments** for projects.
10. **Never share API keys.**



Resources Mentioned

- LangChain Docs: python.langchain.com
- OpenAI Platform: platform.openai.com
- Google AI Studio: aistudio.google.com
- Grok Cloud: console.grok.com
- Mistral AI Studio: console.mistral.ai
- Hugging Face: huggingface.co
- UV Package Manager: docs.astral.sh/uv



Real-World Project: Sinés Analytics

Problem Statement:

- Sinés Analytics is a media intelligence company.
- Helps streaming platforms (Netflix, YouTube, Amazon) organize and analyze movie data.
- Receives unstructured data: movie descriptions, press releases, blogs, reviews.
- Needs **structured movie metadata**.

Solution:

- Build an AI-powered tool that:
 1. Takes a raw paragraph about a movie.
 2. Extracts structured information (title, year, genre, director, cast, etc.).
 3. Generates a clean summary.
 4. Returns it in **JSON format**.

Tech Stack:

- LangChain
- Mistral AI (free model)
- Pydantic (for schema validation)
- Streamlit (for UI)
- Prompt Templates

Workflow:

1. User pastes a movie paragraph.
2. Prompt Template sends it to LLM with system instructions.
3. LLM extracts information.
4. Pydantic Parser validates the output.
5. Structured JSON is returned.



Conclusion

This video laid the **foundation** for Generative AI:

- Understanding LLMs, embeddings, prompts, chains, memory, indexes, agents.
- Setting up the development environment.
- Getting API keys for multiple providers.
- Building your first chatbot with memory and UI.
- Using prompt templates and structured output (Pydantic).

Next up: RAG Systems — connecting external data to LLMs.

"Finally, we are starting Generative AI, guys!" — Aakash Vyas

End of Notes 📝



सो हेलो एंड वेलकम बैक टू द ...
TXT 564.95KB

make notes for me out of this transcript , explain everything properly
deeply

Generative AI Series — Complete Notes (Video 2: RAG Systems)



Introduction

Instructor: Aakash Vyas | **Channel:** Sheryians AI School

This is the **second video** of the Generative AI series. In this video, we complete **RAG (Retrieval Augmented Generation)** applications.

What You'll Learn:

- Document Loaders
- Text Splitters (Chunking)

- Vector Databases
- Retrievers
- **Project:** Build "Coursemate AI" — an AI-powered study assistant

Prerequisites (from Video 1):

- LLM Foundations
- How LLMs work
- API keys
- Chat models
- Hugging Face & local models
- Messages & roles
- Prompts & structured output
- Project: Sinés Analytics



The Project: Coursemate AI

Problem Statement:

Coursemate AI is an AI-powered study assistant that helps students interact with their learning materials more efficiently.

- Modern students rely on **multiple sources**: lecture notes, textbooks, PDFs, research papers
- These documents are **long and difficult to navigate**
- Finding specific information is **time-consuming**

Solution:

- Students can **chat with their study materials**
- Instead of manually searching through pages, students can **ask questions** and receive **accurate answers** extracted directly from their documents.

How It Works (RAG):

- Combines **document retrieval** with **Large Language Models**
- Provides **context-aware explanations, summaries, and answers** from the student's own study resources.

What is RAG? (Recap & Deep Dive)

The Core Idea:

- You have an **LLM** with general knowledge (trained on world data).
- But the LLM **doesn't know your company's data or your private PDFs**.
- **RAG** = Connect external data sources to your LLM → Create a pipeline → Give context-aware answers.

Visual Flow:

text

User PDF → Document Loader → Text Splitter (Chunks) → Embeddings → Vector Database

↓

User Query → Embedding → Similarity Search → Retrieve Relevant Chunks → LLM → Answer



Development Plan (Step-by-Step)

Step	What Happens
1	User uploads study material (PDF, notes, textbook, research paper)
2	System loads document using Document Loaders
3	Text Splitting (Chunking) — break into smaller chunks
4	Embedding Generation — convert chunks to vectors
5	Store in Vector Database

Step	What Happens
6	User asks a question → Query embedding created
7	Similarity Search in vector DB → Retrieve top relevant chunks
8	Retriever selects top chunks → Forms context
9	Context + Query → Sent to LLM → Answer generated

Step 1: Document Loaders

What is a Document Loader?

- Converts **raw files** (PDF, text, web pages) into **Document objects** that can be processed.
- Document object has two things:
 - **page_content** — The actual text
 - **metadata** — Source, page number, author, etc.

Types of Document Loaders:

- **TextLoader** — For .txt files
- **PyPDFLoader** — For PDF files
- **WebBaseLoader** — For websites
- **CSVLoader** — For CSV files
- **UnstructuredLoader** — For various formats
- **Cloud providers** — AWS S3, Azure, Dropbox
- **Social media** — Twitter, Reddit (requires API)
- **Messaging** — Telegram, WhatsApp, Discord (requires API)

Example: Loading a Text File

```
python
```

```
from langchain_community.document_loaders import TextLoader

data = TextLoader("document_loaders/notes.txt")
docs = data.load()
print(docs)
```

Output:

- A list of Document objects
- Each Document has `metadata` (source) and `page_content` (text)

Example: Loading a PDF

python

```
from langchain_community.document_loaders import PyPDFLoader

data = PyPDFLoader("document_loaders/GRU.pdf")
docs = data.load()
print(len(docs)) # Number of pages = number of documents
print(docs[0].page_content)
print(docs[0].metadata)
```

Key Point:

- A PDF with 15 pages creates **15 Document objects** (one per page).
- A text file creates **1 Document object**.

Example: Loading a Website

python

```
from langchain_community.document_loaders import WebBaseLoader

data = WebBaseLoader("https://www.apple.com/mac/")
docs = data.load()
print(docs[0].page_content)
```

Using Loaded Documents with LLM

python

```
from dotenv import load_dotenv
load_dotenv()

from langchain_mistralai import ChatMistralAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_community.document_loaders import TextLoader

# Load document
data = TextLoader("document_loaders/notes.txt")
docs = data.load()

# Create prompt template
template = ChatPromptTemplate.from_messages([
    ("system", "You are an AI that summarizes the text."),
    ("human", "{data}")
])

# Create model
model = ChatMistralAI(model="mistral-small-2506")

# Create final prompt
final_prompt = template.invoke({"data": docs[0].page_content})

# Get response
response = model.invoke(final_prompt)
print(response.content)
```

Problem: Context window limit — can't send entire large PDF at once.

Step 2: Text Splitting (Chunking)

Why Chunking?

1. **Context Window Limit** — LLMs have a maximum token limit.
2. **Better Retrieval in RAG** — Smaller chunks = more precise embeddings.
3. **More Accurate Embeddings** — One clear idea per chunk.
4. **Faster Processing** — Smaller chunks = faster embedding creation.
5. **Faster Similarity Search** — Vector DB search is faster with smaller chunks.

Problem: Large Document

- Example: "Fundamentals of Deep Learning" PDF = **534 pages** → 534 documents
- Can't feed all at once to LLM.

Solution: Split into Chunks

Types of Text Splitting:

1. Character-Based Splitting

- Splits based on **number of characters**.
- Doesn't understand words, tokens, or meaning.
- **Example:** `CharacterTextSplitter` with `chunk_size=10`

python

```
from langchain_text_splitters import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=10,
    chunk_overlap=1,
    separator="" # Empty = split on any character
)

chunks = splitter.split_documents(docs)
print(len(chunks))
for chunk in chunks:
    print(chunk.page_content)
    print("----")
```

Note: LangChain updates frequently — `separator` may replace `splitter` in newer versions.

2. Token-Based Splitting

- Splits based on **number of tokens** (not characters).
- LLMs process tokens, so this aligns better.
- Uses **tiktoken** (OpenAI's tokenizer).

python

```
from langchain_text_splitters import TokenTextSplitter

splitter = TokenTextSplitter(
    chunk_size=100,
    chunk_overlap=10
)

chunks = splitter.split_documents(docs)
print(len(chunks))
```

Why Token-Based?

- More accurate for LLM context windows.
- `tiktoken` is accurate for OpenAI models.

3. Recursive Character Text Splitting (Best for RAG)

- **Most recommended** for RAG applications.
- Tries to split on:
 1. Double newline (`\n\n`)
 2. Single newline (`\n`)
 3. Space ()
 4. Any character
- Preserves meaning and structure.

python

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)

chunks = splitter.split_documents(docs)
print(len(chunks))
```

How It Works:

- First tries to split on `\n\n` (paragraphs)
- If chunk too big, tries `\n` (lines)

- If still too big, tries spaces
- Finally splits on any character

Example:

python

```
# Input text: "Hello how are you\nAnd what is your age\nAnd what are your dreams that you want to achieve"
```

```
# With chunk_size=10, it will split intelligently:
```

```
# Chunk 1: "Hello how are you"
```

```
# Chunk 2: "And what is your age"
```

```
# Chunk 3: "And what are your dreams that you want to achieve"
```

4. Semantic Meaning-Based Splitting (Advanced)

- Splits based on **meaning and structure**.
- Converts sentences to embeddings.
- Compares similarity between consecutive sentences.
- If similarity > threshold → combine; else → split.
- **Still under development** — not widely used yet.



Step 3: Vector Databases

What is a Vector Database?

- Stores **embeddings** (vectors) + original text + metadata.
- Enables **similarity search** — finding similar vectors.

Why Not Normal Databases (MySQL, PostgreSQL, MongoDB)?

- Normal DBs use **exact matches** and **indexing** (B-tree, hash).
- Vector DBs use **Approximate Nearest Neighbor (ANN)** algorithms.
- Vector search works on **semantic similarity**, not exact matches.

How Normal Databases Work:

- Data stored in structured format (rows, columns).
- Search by exact match or indexed column.
- **Example:** `WHERE movie = 'Interstellar'` → Returns exact match.

How Vector Databases Work:

- Data stored as **embeddings** (512, 768, 1536 dimensions).
- Search by **similarity** (cosine similarity, Euclidean distance).
- **Example:** "Movies about space travel" → Finds semantically similar content.

The Problem with Normal Search:

- 1 lakh embeddings × 512 dimensions each.
- Comparing query embedding with all 1 lakh embeddings = **$O(N)$ complexity**.
- Too slow for large datasets.

How Vector DBs Solve This:

- Use **ANN algorithms** like:
 - **HNSW** (Hierarchical Navigable Small World)
 - **IVF** (Inverted File Index)
 - **PQ** (Product Quantization)
- **Example: IVF**
 1. Divide 1 lakh embeddings into 5 clusters (using K-Means).
 2. Each cluster has 20,000 embeddings.
 3. Calculate **centroid** (average) of each cluster.
 4. Query → Compare with 5 centroids → Find best cluster.
 5. Search only in that cluster (20,000 embeddings).
 6. **Result:** 5x faster search.

Popular Vector Databases:

Vector DB	Notes
Chroma DB	Easiest, local, good for learning
FAISS	Facebook AI, fast
Pinecone	Production-level, managed
Weaviate	Open source
Qdrant	Production-level
Milvus	Scalable
PGVector	PostgreSQL extension
MongoDB Atlas	Vector search in MongoDB

Using Chroma DB (Example):

python

```
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document

# Create documents
docs = [
    Document(page_content="Python is widely used in AI.", metadata={"source": "AI Book"}),
    Document(page_content="Pandas is used for data analysis in Python.", metadata={"source": "Data Science Book"}),
    Document(page_content="Neural networks are used in deep learning.", metadata={"source": "Deep Learning Book"})
]

# Create embedding model
embedding = OpenAIEmbeddings()

# Create vector store
vector_store = Chroma.from_documents(
    documents=docs,
    embedding=embedding,
    persist_directory="chroma_db"
)
```

```
# Similarity search
results = vector_store.similarity_search("What is used for data analysis?", k=2)
for r in results:
    print(r.page_content)
    print(r.metadata)
```

Output:

- "Pandas is used for data analysis in Python."
- "Neural networks are used in deep learning."

Chroma DB File Structure:

- `chroma.sqlite3` — SQLite database storing metadata, page content
- `data_level0.bin`, `header.bin`, `length.bin`, `link_lists.bin` — Embedding storage
- **Note:** Chroma DB uses SQLite for structured storage + binary files for embeddings.

Retrieving from Chroma DB:

python

```
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

# Load existing vector store
vector_store = Chroma(
    persist_directory="chroma_db",
    embedding_function=OpenAIEmbeddings()
)

# Create retriever
retriever = vector_store.as_retriever()

# Retrieve documents
docs = retriever.invoke("What is used for data analysis?")
for doc in docs:
    print(doc.page_content)
    print(doc.metadata)
```

Step 4: Retrievers

What is a Retriever?

- Takes a **user query** and returns the **most relevant documents/chunks** from the database.
- **NOT** responsible for answering — only retrieving.
- LLM is responsible for answering.

Two Types of Retrievers:

1. By Data Source

- Fetches information from **external sources** (APIs, datasets).
- **Examples:**
 - **ArXiv Retriever** — Research papers
 - **Wikipedia Retriever** — Wikipedia articles
 - **PubMed Retriever** — Medical papers
- Connects to APIs and returns relevant documents.

Example: ArXiv Retriever

```
python

from langchain_community.retrievers import ArXivRetriever

retriever = ArXivRetriever(
    load_max_docs=2,
    load_all_available_meta=True
)

docs = retriever.invoke("Large Language Models")
for doc in docs:
    print(doc.metadata["title"])
    print(doc.metadata["author"])
    print(doc.page_content[:500])
```

2. By Retrieval Strategy

- Uses **your own vector store**.

- Multiple search strategies available:
 1. **Similarity Search** (default)
 2. **MMR (Maximum Marginal Relevance)**
 3. **Multi-Query Retrieval**

Retrieval Strategy 1: Similarity Search

How It Works:

- Compares query vector with document vectors.
- Uses **cosine similarity**, **Euclidean distance**, etc.
- Returns **top K most similar** documents.

Example:

```
python

retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3}
)

docs = retriever.invoke("What is gradient descent?")
for doc in docs:
    print(doc.page_content)
```

Problem:

- Returns **redundant** results (similar chunks saying the same thing).
- Wastes context window & tokens.
- Lacks **diversity**.

Example:

- Query: "What is gradient descent?"
- Returns:
 - Chunk 1: "Gradient descent is an optimization algorithm used in ML."
 - Chunk 2: "Gradient descent minimizes the loss function."

- Chunk 3: "Gradient descent is an optimization that minimizes loss."
- Chunks 1, 2, 3 are **redundant**.

Retrieval Strategy 2: MMR (Maximum Marginal Relevance)

Goal:

- Balance **relevance** and **diversity**.
- Returns documents that are **relevant but different** from each other.

How It Works:

1. Retrieve top **fetch_k** documents (e.g., 10) using similarity.
2. Apply MMR to select **k** documents (e.g., 4) that are diverse.
3. **Lambda** parameter (0 to 1):
 - `lambda_mult=1` → Only relevance (no diversity)
 - `lambda_mult=0` → Maximum diversity
 - `lambda_mult=0.5` → Balanced

Example:

python

```
retriever = vector_store.as_retriever(  
    search_type="mmr",  
    search_kwargs={  
        "k": 4,  
        "fetch_k": 10,  
        "lambda_mult": 0.5  
    }  
)  
  
docs = retriever.invoke("What is gradient descent?")  
for doc in docs:  
    print(doc.page_content)
```

Result:

- Chunk 1: "Gradient descent is an optimization algorithm used in ML."

- Chunk 2: "Gradient descent minimizes the loss function."
- Chunk 3: "Support vector machines are supervised learning algorithms."
- **Different topics** = better context.

Retrieval Strategy 3: Multi-Query Retrieval

Problem:

- A single query might not capture all aspects of a concept.
- **Example:** "What is gradient descent?" is too narrow.

Solution:

- Use **LLM** to generate **multiple variations** of the query.
- Search for each variation.
- Combine results.

How It Works:

1. User query: "What is gradient descent?"
2. LLM generates:
 - "Explain gradient descent."
 - "How does gradient descent work?"
 - "Optimization algorithm used in neural networks."
3. Retriever searches for each query.
4. Combines all results.

Example:

python

```
from langchain_classic.retrievers.multi_query import MultiQueryRetriever
from langchain_mistralai import ChatMistralAI

# Create LLM for query generation
llm = ChatMistralAI(model="mistral-small-2506")

# Create base retriever
base_retriever = vector_store.as_retriever()
```

```

# Create multi-query retriever
multi_query_retriever = MultiQueryRetriever.from_llm(
    retriever=base_retriever,
    llm=llm
)

# Retrieve
docs = multi_query_retriever.invoke("What is gradient descent?")
for doc in docs:
    print(doc.page_content)

```

Note: LangChain updates frequently — import paths may change (e.g., `langchain_classic` VS `langchain`).



Building the Full RAG Application

Step 1: Create Database (One-Time)

File: `create_database.py`

python

```

from dotenv import load_dotenv
load_dotenv()

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# 1. Load PDF
loader = PyPDFLoader("document_loaders/deep_learning.pdf")
docs = loader.load()

# 2. Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(docs)

# 3. Create embeddings

```

```
embedding_model = OpenAIEmbeddings()

# 4. Store in Chroma DB
vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embedding_model,
    persist_directory="chroma_db"
)

print("Database created successfully!")
```

Step 2: Main Application (RAG Pipeline)

File: main.py

```
python

from dotenv import load_dotenv
load_dotenv()

from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_mistralai import ChatMistralAI
from langchain_core.prompts import ChatPromptTemplate

# 1. Load embedding model
embedding_model = OpenAIEmbeddings()

# 2. Load existing vector store
vector_store = Chroma(
    persist_directory="chroma_db",
    embedding_function=embedding_model
)

# 3. Create retriever (MMR strategy)
retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={
        "k": 4,
        "fetch_k": 10,
        "lambda_mult": 0.5
    }
)
```

```

# 4. Create LLM
llm = ChatMistralAI(model="mistral-small-2506")

# 5. Create Prompt Template
prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a helpful AI assistant.
    Use only the provided context to answer the question.
    If the answer is not present in the context, say "I could not find the answer in the
    document."

    Context: {context}"""),
    ("human", "{question}")
])

# 6. Chat Loop
print("RAG System Created")
print("Press 0 to exit")

while True:
    query = input("You: ")
    if query == "0":
        break

    # Retrieve relevant documents
    docs = retriever.invoke(query)

    # Create context
    context = "\n".join([doc.page_content for doc in docs])

    # Create final prompt
    final_prompt = prompt.invoke({
        "context": context,
        "question": query
    })

    # Get response
    response = llm.invoke(final_prompt)

    print(f"\nAI: {response.content}\n")

```

Step 3: Streamlit UI

File: app.py

python

```
import streamlit as st
from dotenv import load_dotenv
load_dotenv()

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_mistralai import ChatMistralAI
from langchain_core.prompts import ChatPromptTemplate
import os
import shutil

st.title("RAG Book Assistant")
st.write("Upload a PDF and ask questions from the document.")

# File uploader
uploaded_file = st.file_uploader("Upload the PDF book", type=["pdf"])

if uploaded_file:
    # Save uploaded file
    with open("uploaded_book.pdf", "wb") as f:
        f.write(uploaded_file.getbuffer())

    st.success("PDF uploaded successfully!")

    # Create Vector Database button
    if st.button("Create Vector Database"):
        # Delete existing Chroma DB
        if os.path.exists("chroma_db"):
            shutil.rmtree("chroma_db")

        with st.spinner("Processing document..."):
            # Load PDF
            loader = PyPDFLoader("uploaded_book.pdf")
            docs = loader.load()

            # Split
            splitter = RecursiveCharacterTextSplitter(
                chunk_size=1000,
                chunk_overlap=200
            )
            chunks = splitter.split_documents(docs)

            # Embeddings
```

```

embedding = OpenAIEmbeddings()

# Vector Store
vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embedding,
    persist_directory="chroma_db"
)

st.success("Vector Database created!")

# Question Answering
st.subheader("Ask Questions from the Book")
query = st.text_input("Enter your question:")

if query:
    with st.spinner("Searching..."):
        # Load vector store
        embedding = OpenAIEmbeddings()
        vector_store = Chroma(
            persist_directory="chroma_db",
            embedding_function=embedding
        )

        # Retriever
        retriever = vector_store.as_retriever(
            search_type="mmr",
            search_kwargs={"k": 4, "fetch_k": 10, "lambda_mult": 0.5}
        )

        # Retrieve
        docs = retriever.invoke(query)
        context = "\n".join([doc.page_content for doc in docs])

        # LLM
        llm = ChatMistralAI(model="mistral-small-2506")

        # Prompt
        prompt = ChatPromptTemplate.from_messages([
            ("system", """"You are a helpful AI assistant.
            Use only the provided context to answer.
            If not found, say "I could not find the answer in the document."

            Context: {context}"""),
            ("human", "{question}")
        ])

```

```
final_prompt = prompt.invoke({
    "context": context,
    "question": query
})

response = llm.invoke(final_prompt)

st.write("**AI Answer:**")
st.write(response.content)
```

Run the app:

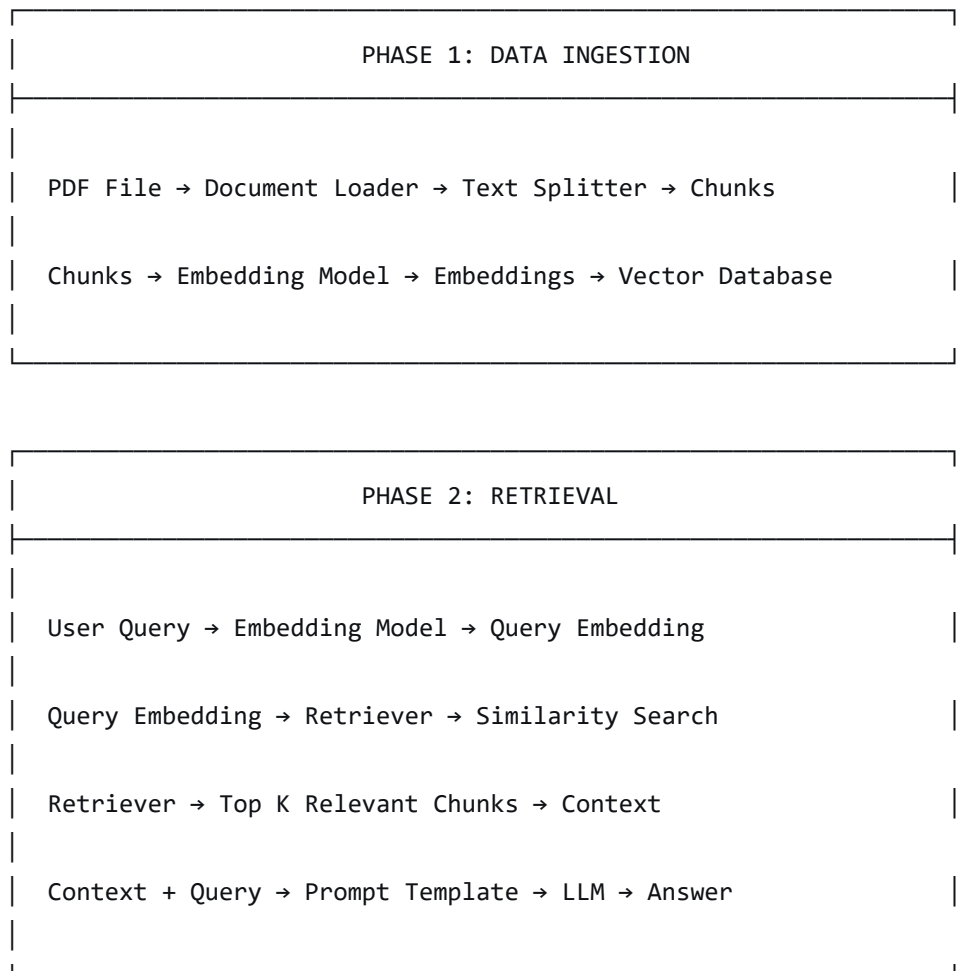
```
bash
```

```
streamlit run app.py
```



Complete RAG Architecture

text



Key Concepts Summary

Concept	What It Is	Why It Matters
Document Loader	Converts files to Document objects	Foundation of RAG
Text Splitter	Breaks large docs into chunks	Fits context window
Chunk Size	Number of characters/tokens per chunk	Balance detail vs. context
Chunk Overlap	Overlap between chunks	Preserves meaning across boundaries
Embedding	Text → Numbers (vectors)	Enables semantic search
Vector Database	Stores embeddings + text + metadata	Fast similarity search
Retriever	Fetches relevant chunks	Bridge between query and LLM
Similarity Search	Compares vectors by distance	Basic retrieval
MMR	Relevance + Diversity	Avoids redundant chunks
Multi-Query	Generates query variations	Captures all aspects
RAG	Retrieval Augmented Generation	Context-aware answers
Chroma DB	Local vector database	Easy to learn
Cosine Similarity	Angle between vectors	Common similarity metric
Euclidean Distance	Straight-line distance	Another similarity metric
IVF	Inverted File Index	Speeds up search (clustering)
HNSW	Hierarchical Navigable Small World	Graph-based ANN search

LangChain Components Used

python

Document Loaders

```
from langchain_community.document_loaders import (  
    TextLoader,          # .txt files  
    PyPDFLoader,         # PDF files  
    WebBaseLoader,       # Websites  
    CSVLoader            # CSV files  
)
```

Text Splitters

```
from langchain_text_splitters import (  
    CharacterTextSplitter,    # Split by characters  
    TokenTextSplitter,       # Split by tokens  
    RecursiveCharacterTextSplitter # Best for RAG  
)
```

Embeddings

```
from langchain_openai import OpenAIEmbeddings  
from langchain_community.embeddings import HuggingFaceEmbeddings
```

Vector Stores

```
from langchain_community.vectorstores import (  
    Chroma,      # Local, easy  
    FAISS,       # Facebook AI  
    Pinecone,    # Managed  
    Qdrant       # Production  
)
```

Retrievers

```
from langchain_community.retrievers import ArXivRetriever  
from langchain_classic.retrievers.multi_query import MultiQueryRetriever
```

Models

```
from langchain_mistralai import ChatMistralAI  
from langchain_openai import ChatOpenAI  
from langchain_google_genai import ChatGoogleGenerativeAI
```

Prompts

```
from langchain_core.prompts import ChatPromptTemplate
```

Messages

```
from langchain_core.messages import (  
    HumanMessage,  
    AIMessage,
```

```
        SystemMessage
    )
```

Important Notes

LangChain Updates Frequently

- Import paths change (e.g., `langchain_classic` VS `langchain`)
- Method names change (e.g., `splitter` → `separator`)
- Always check **documentation** for latest syntax.
- Use ChatGPT to debug errors.

API Keys Required

```
env
```

```
OPENAI_API_KEY=your_key_here
MISTRAL_API_KEY=your_key_here
GOOGLE_API_KEY=your_key_here
HUGGINGFACEHUB_ACCESS_TOKEN=your_token_here
```

Virtual Environment

```
bash
```

```
uv venv
source .venv/bin/activate  # Mac/Linux
.venv\Scripts\activate     # Windows
uv pip install -r requirements.txt
```

Requirements File

```
text
```

```
langchain
langchain-core
langchain-community
langchain-openai
```

langchain-mistralai
langchain-google-genai
langchain-huggingface
chromadb
pypdf
sentence-transformers
streamlit
python-dotenv
tiktoken

Real-World Example: Query "What is Word2Vec?"

1. **User asks:** "Can you tell me about the Word2Vec framework?"
2. **Query embedding** created.
3. **Retriever** searches vector DB.
4. **Top 4 chunks** retrieved (using MMR).
5. **Context** created from chunks.
6. **Final prompt** = Context + Query.
7. **LLM** generates answer: "The Word2Vec framework is a method for generating word embeddings, pioneered by Mikolov et al. It uses two strategies: CBOW and Skip-gram..."
8. **Answer matches** page 179 of the PDF.

Pro Tips

1. **Always use RecursiveCharacterTextSplitter** for RAG.
2. **Chunk size 1000, overlap 200** is a good starting point.
3. **Use MMR** to avoid redundant chunks.
4. **Use Multi-Query** for better coverage.
5. **Chroma DB** for learning, **Pinecone/Qdrant** for production.
6. **Hugging Face embeddings** are free alternatives to OpenAI.
7. **Delete Chroma DB** when uploading a new book (if you want only new content).
8. **Test with different K values** (3, 4, 5) to find optimal retrieval.

9. **Lambda in MMR**: 0 = max diversity, 1 = max relevance.
10. **Cache embeddings** to avoid repeated API costs.



What's Next?

Video 3: Agentic AI

- **Runnables (Chains)** — Deep dive
- **Tools** — How agents use tools
- **Agentic Systems** — Multiple AI agents working together
- **LangGraph** — Building agent workflows

Video 4: Big Project with Deployment

- Full project with deployment
- Scalable models
- Production-level RAG



Resources

- **LangChain Docs**: python.langchain.com
- **Chroma DB**: docs.trychroma.com
- **OpenAI Embeddings**: platform.openai.com/docs/guides/embeddings
- **Hugging Face**: huggingface.co
- **ArXiv**: arxiv.org
- **Streamlit**: streamlit.io

❏ Conclusion

In this video, we:

1. Understood **RAG** and its importance.

2. Learned **Document Loaders** (Text, PDF, Web).
3. Learned **Text Splitting** (Character, Token, Recursive).
4. Learned **Vector Databases** (Chroma DB, similarity search, ANN algorithms).
5. Learned **Retrievers** (Similarity, MMR, Multi-Query).
6. Built **Coursemate AI** — a complete RAG application.
7. Created a **Streamlit UI** for the application.

Next: Runnables, Tools, and Agentic AI.

"Finally, we are completing our RAG applications!" — Aakash Vyas

End of Notes 📄



सो हेलो एंड वेलकम बैक टू शे...
TXT 648.15KB

make notes for me out of this transcript , explain everything properly deeply

Generative AI Series — Complete Notes (Video 3: Runnables, Tools & Agents)

Introduction

Instructor: Aakash Vyas | **Channel:** Sheryians AI School

This is the **third video** of the Generative AI series. In this video, we complete **Runnables, Tools, and Agents**.

Prerequisites:

- Video 1 (LLM Foundations)
- Video 2 (RAG Systems)

What You'll Learn:

Topic	Description
Runnables	The new LangChain abstraction (replaces Chains)
Tools	Built-in & Custom tools
Agents	AI systems that use tools autonomously
Projects	Multiple small projects + City Intelligence Agent

Table of Contents:

1. **Runnables** — Sequences, Parallel, Lambda, Pass-through
2. **Tools** — Built-in (Tavily) & Custom tools
3. **Tool Calling** — Binding, Calling, Execution
4. **Agents** — Manual (Human-in-the-Loop) & Automated (create_agent)
5. **Project** — City Intelligence Agent



Part 1: Runnables

The Story: Why Runnables?

Problem 1: Different SDKs

- Earlier, each LLM provider (OpenAI, Anthropic, Google) had **different SDKs**.
- Switching providers meant **rewriting a large part of your code**.

Solution: LangChain introduced a **Unified Interface**.

- Same code works for any LLM.
- Just change the model name — everything else stays the same.

Problem 2: Multiple Components = Manual Work

As applications grew, they needed:

- LLM
- Prompt Template

- Output Parser
- Retriever
- Embedding Model
- Vector Store
- Custom Logic

Each had **different functions**:

- `prompt.format_messages()`
- `model.predict_messages()`
- `parser.parse()`

Problem: You had to **manually call each component** in the correct sequence.

- Repetitive
- Error-prone
- Messy code
- Hard to maintain

Solution 1: Chains (Old)

- Connected components using a **pipe operator** (`|`).
- Output of one component becomes input of the next.
- **Example:** `prompt | model | parser`

But Chains had problems:

- **50+ different types of chains** (LLMChain, SequentialChain, RouterChain, RetrievalQAChain, etc.)
- Confusing — which chain to use?
- Not flexible — couldn't add custom logic between steps.
- No parallel execution.
- Limited control over data flow.

Solution 2: Runnables (New)

- **Everything becomes a Runnable.**
- Prompt Template → Runnable

- Model → Runnable
- Output Parser → Runnable
- Retriever → Runnable
- Even your **own Python function** → Runnable

Key Insight:

Think of components as **LEGO blocks**. Each block has a different purpose, but they all **connect the same way**.

All Runnables expose the same methods:

- `.invoke()` — Single input → Output
- `.batch()` — Multiple inputs → Multiple outputs
- `.stream()` — Token-by-token output

Note: In LangChain v2, Chains disappeared. In v3, Chains are back but work with Runnables.

Types of Runnables

1. Sequence Runnable (Basic Pipeline)

What it does: Connects components one after another.

- Output of Component 1 → Input of Component 2
- Output of Component 2 → Input of Component 3

Example:

python

```
from langchain_mistralai import ChatMistralAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Components
prompt = ChatPromptTemplate.from_template("Explain {topic} in simple words")
model = ChatMistralAI(model="mistral-small-2506")
parser = StrOutputParser()
```

```
# Chain (Sequence Runnable)
chain = prompt | model | parser

# Invoke
result = chain.invoke({"topic": "Machine Learning"})
print(result)
```

Flow:

1. User gives topic → Prompt creates formatted prompt
2. Formatted prompt → Model generates response
3. Response → Parser cleans it up
4. Final result saved

Key Point: No manual calling — everything is connected automatically.

2. Parallel Runnable

What it does: Runs **multiple pipelines simultaneously** on the same input.

When to use:

- You want multiple outputs at the same time.
- Example: Short explanation + Detailed explanation.
- Example: Same answer in different formats.

How it works:

- Define multiple pipelines inside a **dictionary**.
- All pipelines run **simultaneously**.
- Each pipeline produces its own output.
- Results come back in a **structured format**.

Example:

```
python

from langchain_core.runnables import RunnableParallel

# Pipeline 1: Short explanation
```

```

short_prompt = ChatPromptTemplate.from_template("Explain {topic} in 1-2 lines")
short_chain = short_prompt | model | parser

# Pipeline 2: Detailed explanation
detailed_prompt = ChatPromptTemplate.from_template("Explain {topic} in detail")
detailed_chain = detailed_prompt | model | parser

# Parallel Runnable
chain = RunnableParallel(
    short=short_chain,
    detailed=detailed_chain
)

# Invoke
result = chain.invoke({"topic": "Machine Learning"})
print(result["short"])
print(result["detailed"])

```

Result:

- Both pipelines run **at the same time**.
- `result` is a dictionary with `short` and `detailed` keys.

Why Parallel?

- Saves time (both run simultaneously).
- More efficient than running one after another.

3. Runnable Lambda

What it does: Inserts a **custom Python function** inside a pipeline.

When to use:

- You want to modify data between steps.
- You want to extract a specific part of a dictionary.
- You want to add custom logic.

Example:

```
python
```

```

from langchain_core.runnables import RunnableLambda

# Custom function
def extract_short(x):
    return x["short"]

# Pipeline
chain = RunnableParallel(
    short=short_chain,
    detailed=detailed_chain
) | RunnableLambda(lambda x: x["short"])

result = chain.invoke({"topic": "Machine Learning"})
print(result)

```

How it works:

- `RunnableLambda` takes a function.
- The function receives the input.
- Whatever the function returns becomes the output.

Real-world use case:

- Multiple inputs in a dictionary.
- Each pipeline needs a different input.
- Use `RunnableLambda` to extract the right part.

Example with different inputs:

python

```

chain = RunnableParallel(
    short=RunnableLambda(lambda x: x["short"]) | short_chain,
    detailed=RunnableLambda(lambda x: x["detailed"]) | detailed_chain
)

result = chain.invoke({
    "short": "Machine Learning",
    "detailed": "Deep Learning"
})

```

4. Runnable Pass-Through

What it does: Passes the input **unchanged** to the output.

Example:

python

```
from langchain_core.runnables import RunnablePassthrough

chain = RunnablePassthrough()
result = chain.invoke("Hello")
print(result) # Output: "Hello"
```

When to use:

- You want to **preserve** the original input.
- You want to use it alongside other outputs in later steps.
- Example: You want both the code AND its explanation.

Real-world Example: Code Generator + Explainer

Problem: You want an AI that:

1. Generates code
2. Explains the code
3. Returns both

Solution:

python

```
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# Step 1: Generate code
code_chain = code_prompt | model | parser

# Step 2: Parallel - keep code + explain it
chain = RunnableParallel(
    code=RunnablePassthrough(), # Keep original code
    explanation=explain_prompt | model | parser # Explain it
)

# Full pipeline
```

```
full_chain = code_chain | chain

result = full_chain.invoke({"topic": "Write a palindrome code in Python"})
print(result["code"])
print(result["explanation"])
```

Why Pass-Through?

- Without it, you'd lose the original code after explaining it.
- With it, you keep both the code AND the explanation.

Summary: Runnables

Runnable	Purpose
Sequence	Connect components one after another
Parallel	Run multiple pipelines simultaneously
Lambda	Insert custom Python function
Pass-Through	Preserve original input

Key Benefits:

- Same methods (`.invoke()` , `.batch()` , `.stream()`)
- Same SDK for all components
- Flexible — add custom logic anywhere
- Clean, maintainable code

Part 2: Tools

Why Tools?

LLMs are powerful but have limitations:

Limitation	Example
No real-time data	Doesn't know today's news
No accurate calculations	Struggles with complex math
No API access	Can't call external APIs
No personal data	Doesn't know your company's data
No actions	Can't send emails, book tickets, etc.

Solution: Tools

LLM = Brain. Tools = Hands & Legs.

Without tools, an LLM is like a brain without a body — it can think but can't **do** anything.

Tools = Functions + External Services that LLMs can use to perform real-world tasks.

Types of Tools

1. Built-in Tools

What they are: Pre-built tools provided by LangChain.

Examples:

- **Search Tools:** Bing, Brave, DuckDuckGo, Google, Tavily
- **Code Interpreters:** Amazon Bedrock, Azure, Riza
- **Productivity:** Gmail, Slack, Infobip
- **Web Browsing:** AgentQL, Hyperbrowser

Example: Tavily Search (for AI News Curator)

```
python

from langchain_community.tools.tavily_search import TavilySearchResults

# Create search tool
search_tool = TavilySearchResults(max_results=5)
```

```
# Use the tool
results = search_tool.invoke("Latest AI news of 2026")
print(results)
```

How to get API Key:

1. Go to Tavily API Keys
2. Sign up / Log in
3. Copy API key
4. Store in .env :

text

```
TAVILY_API_KEY=your_key_here
```

Full Example: AI News Summarizer

python

```
from dotenv import load_dotenv
load_dotenv()

from langchain_mistralai import ChatMistralAI
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Create search tool
search_tool = TavilySearchResults(max_results=5)

# Create LLM
llm = ChatMistralAI(model="mistral-small-2506")

# Create prompt
prompt = ChatPromptTemplate.from_template(
    "You are a helpful assistant. Summarize the following news into clear bullet points:\n\n{news}"
)

# Create chain
chain = prompt | llm | StrOutputParser()

# Get news
news_result = search_tool.invoke("Latest AI news of 2026")
```

```
# Summarize
result = chain.invoke({"news": news_result})
print(result)
```

2. Custom Tools

What they are: Tools you create yourself based on your needs.

Examples:

- Access your database
- Call your own API
- Process your data
- Custom calculations

How to create a Custom Tool:

Step 1: Create a Python function

python

```
def get_greeting(name: str) -> str:
    """Generate a greeting message for a user."""
    return f"Hello {name}, welcome to the AI world!"
```

Step 2: Add @tool decorator

python

```
from langchain.tools import tool

@tool
def get_greeting(name: str) -> str:
    """Generate a greeting message for a user."""
    return f"Hello {name}, welcome to the AI world!"
```

Step 3: Use the tool

python

```
# Tools are Runnables – use .invoke()
result = get_greeting.invoke({"name": "Aakash"})
print(result) # Output: Hello Aakash, welcome to the AI world!
```

Key Components of a Tool:

Component	Purpose
Function	The actual logic
Type Hints	Input/output data types
Docstring	Tells the LLM what the tool does
@tool decorator	Converts function into a Tool

Tool Properties:

```
python

print(get_greeting.name)           # "get_greeting"
print(get_greeting.description)    # "Generate a greeting message for a user."
print(get_greeting.args)           # {"name": {"title": "Name", "type": "string"}}
```

Tool Calling: The 4 Steps

Step 1: Create a Tool

```
python

@tool
def get_text_length(text: str) -> int:
    """Returns the number of characters in a given text."""
    return len(text)
```

Step 2: Bind Tool to LLM

```
python

from langchain_mistralai import ChatMistralAI
```

```
llm = ChatMistralAI(model="mistral-small-2506")  
llm_with_tools = llm.bind_tools([get_text_length])
```

What binding does:

- Gives the tool to the LLM.
- LLM now knows:
 - What tools are available
 - What they do
 - When to use them

Important: When you bind tools, the **entire tool metadata** (name, description, args) is sent to the LLM. This increases **input tokens**.

Step 3: Tool Calling (LLM Suggests Tool)

```
python
```

```
result = llm_with_tools.invoke("Return the number of characters in 'Hello how are you?')")  
print(result.tool_calls)
```

Output:

```
python
```

```
[  
  {  
    "name": "get_text_length",  
    "args": {"text": "Hello how are you?"},  
    "id": "call_abc123",  
    "type": "tool_call"  
  }  
]
```

Key Point:

- LLM **does NOT** execute the tool.
- LLM only **suggests** which tool to call.
- The `tool_calls` list contains the suggestion.

Step 4: Tool Execution (You Execute)

```
python
```

```
# Extract tool name and args
tool_call = result.tool_calls[0]
tool_name = tool_call["name"]
tool_args = tool_call["args"]

# Execute the tool
tool_result = get_text_length.invoke(tool_args)
print(tool_result) # Output: 17
```

Full Flow:

1. **Create Tool** → @tool decorator
2. **Bind Tool** → llm.bind_tools([tool])
3. **Tool Calling** → LLM suggests which tool to use
4. **Tool Execution** → You execute the tool and get the result

Human-in-the-Loop (Manual Tool Execution)

Why?

- You want to **approve** before executing a tool.
- Example: Before booking a ticket, you want confirmation.

How it works:

1. LLM suggests tool call.
2. You ask the user: "Agent wants to call `get_weather` . Approve? (yes/no)"
3. If yes → Execute the tool.
4. If no → Deny and continue.

Example:

```
python
```

```
# LLM suggests tool
result = llm_with_tools.invoke(messages)

# Check if tool call is needed
```

```

if result.tool_calls:
    tool_call = result.tool_calls[0]
    tool_name = tool_call["name"]

    # Ask for confirmation
    confirm = input(f"Agent wants to call {tool_name}. Approve? (yes/no): ")

    if confirm.lower() == "yes":
        # Execute tool
        tool_result = tools[tool_name].invoke(tool_call["args"])

        # Create tool message
        tool_message = ToolMessage(
            content=str(tool_result),
            tool_call_id=tool_call["id"]
        )
        messages.append(tool_message)

        # Get final response
        final_result = llm_with_tools.invoke(messages)
        print(final_result.content)
    else:
        print("Tool call denied by user.")

```

Message History:

- **Human Message** → User's query
- **AI Message** → LLM's tool suggestion
- **Tool Message** → Tool's result
- **AI Message** → Final answer



Part 3: Agents

What is an Agent?

Agent = LLM (Brain) + Tools (Hands) + Control Logic (Code)

Agent vs. Human-in-the-Loop:

- **Human-in-the-Loop:** You manually execute tools.
- **Agent:** Tools are executed **automatically** in a loop.

Example: Planning an Evening

- Check weather → Book cab → Order food → Book movie tickets
- Multiple decisions, multiple tools, multiple steps.
- An agent does all of this **autonomously**.

Manual Agent (Hardcoded Loop)

How it works:

1. Create tools.
2. Bind tools to LLM.
3. Create a loop:
 - LLM suggests tool.
 - Execute tool.
 - Add tool message to history.
 - Repeat until no more tool calls.
4. Get final answer.

Example: City Intelligence Agent (Manual)

python

```
from dotenv import load_dotenv
load_dotenv()

import os
import requests
from langchain_mistralai import ChatMistralAI
from langchain.tools import tool
from langchain_core.messages import HumanMessage, ToolMessage, AIMessage
from tavily import TavilyClient

# Tool 1: Weather
@tool
def get_weather(city: str) -> str:
    """Get current weather of a city."""
    api_key = os.getenv("OPENWEATHER_API_KEY")
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}&units=metric"
```

```

    response = requests.get(url)
    data = response.json()
    weather = data["weather"][0]["description"]
    temp = data["main"]["temp"]
    return f"Weather in {city}: {weather}, {temp}°C"

# Tool 2: News
@tool
def get_news(city: str) -> str:
    """Get latest news about a city."""
    client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))
    query = f"latest news in {city}"
    response = client.search(query=query, search_depth="basic", max_results=3)
    news = "\n\n".join([f"{r['title']}\n{r['url']}" for r in response["results"]])
    return f"Latest news for {city}:\n{news}"

# LLM
llm = ChatMistralAI(model="mistral-small-2506")
llm_with_tools = llm.bind_tools([get_weather, get_news])

# Tools dictionary
tools = {
    "get_weather": get_weather,
    "get_news": get_news
}

# Message history
messages = []

print("City Intelligence System")
print("Type 'exit' to quit.")

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break

    messages.append(HumanMessage(content=user_input))

# Agent Loop
while True:
    result = llm_with_tools.invoke(messages)
    messages.append(result)

    # If no tool calls, break
    if not result.tool_calls:
        print("Final Answer:", result.content)

```

```

        break

    # Execute tools
    for tool_call in result.tool_calls:
        tool_name = tool_call["name"]

        # Human-in-the-loop
        confirm = input(f"Agent wants to call {tool_name}. Approve? (yes/no): ")
        if confirm.lower() != "yes":
            print("Tool call denied.")
            break

        # Execute tool
        tool_result = tools[tool_name].invoke(tool_call["args"])

        # Add tool message
        tool_message = ToolMessage(
            content=str(tool_result),
            tool_call_id=tool_call["id"]
        )
        messages.append(tool_message)

    # Continue loop
    continue

```

Automated Agent (create_agent)

What is `create_agent` ?

- A **high-level abstraction** provided by LangChain.
- Automates everything we did manually.
- Handles:
 - The loop
 - Tool calls
 - Message management
 - Stop conditions

How to use:

Step 1: Import

```
python
```

```
from langchain.agents import create_agent
```

Step 2: Create Agent

```
python
```

```
agent = create_agent(  
    model=llm,  
    tools=[get_weather, get_news],  
    system_prompt="You are a helpful city assistant."  
)
```

Step 3: Invoke

```
python
```

```
result = agent.invoke({  
    "messages": [  
        {"role": "user", "content": "What is the weather in Bhopal?"}  
    ]  
})  
  
print(result["messages"][-1].content)
```

Full Example:

```
python
```

```
from dotenv import load_dotenv  
load_dotenv()  
  
import os  
import requests  
from langchain_mistralai import ChatMistralAI  
from langchain.tools import tool  
from langchain.agents import create_agent  
from tavily import TavilyClient  
  
# Tools  
@tool  
def get_weather(city: str) -> str:  
    """Get current weather of a city."""  
    api_key = os.getenv("OPENWEATHER_API_KEY")
```

```

url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}&units=metric"

response = requests.get(url)
data = response.json()
weather = data["weather"][0]["description"]
temp = data["main"]["temp"]
return f"Weather in {city}: {weather}, {temp}°C"

@tool
def get_news(city: str) -> str:
    """Get latest news about a city."""
    client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))
    query = f"latest news in {city}"
    response = client.search(query=query, search_depth="basic", max_results=3)
    news = "\n\n".join([f"{r['title']}\n{r['url']}" for r in response["results"]])
    return f"Latest news for {city}:\n{news}"

# LLM
llm = ChatMistralAI(model="mistral-small-2506")

# Agent
agent = create_agent(
    model=llm,
    tools=[get_weather, get_news],
    system_prompt="You are a helpful city assistant."
)

# Chat Loop
print("City Agent | Type 'exit' to quit")
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break

    result = agent.invoke({
        "messages": [
            {"role": "user", "content": user_input}
        ]
    })

    print("Bot:", result["messages"][-1].content)

```

Human-in-the-Loop with create_agent

What if you want approval before tool execution?

Use `wrap_tool_call` middleware:

python

```
from langchain.agents.middleware import wrap_tool_call

@wrap_tool_call
def human_approval(request, handler):
    """Ask for human approval before every tool call."""
    tool_name = request.tool_call["name"]

    confirm = input(f"Agent wants to call {tool_name}. Approve? (yes/no): ")

    if confirm.lower() != "yes":
        # Deny the tool call
        return ToolMessage(
            content="Tool call denied by user.",
            tool_call_id=request.tool_call["id"]
        )

    # Approve - execute the tool
    return handler(request)

# Create agent with middleware
agent = create_agent(
    model=llm,
    tools=[get_weather, get_news],
    system_prompt="You are a helpful city assistant.",
    middleware=[human_approval]
)
```

What is Middleware?

- Something you add **in between** an automated process.
- Here, it adds a **human approval step** before tool execution.

Manual vs. Automated Agent

Feature	Manual Agent	Automated Agent (<code>create_agent</code>)
Code	You write the loop	LangChain handles it
Control	Full control over every step	Less control, more automation
Visibility	Every step visible	Hidden behind the scenes
Boilerplate	Lots of repetitive code	Minimal code
Error-prone	Yes (manual loops)	No (handled internally)
Scaling	Hard to scale	Easy to scale
Best for	Learning, debugging	Production, real projects

 Project: City Intelligence Agent

Overview

- **Goal:** Build an agent that gives real-time weather + news for any city.
- **Tools:** OpenWeather API + Tavily Search
- **LLM:** Mistral Small (free)
- **Approach:** Manual loop (Human-in-the-Loop) + Automated (`create_agent`)

Flow:

text

User Query → LLM → Decides which tool(s) to use → Executes tools → Gets results → Generates final answer

Tools:

1. **get_weather(city)** → OpenWeather API → Real-time weather
2. **get_news(city)** → Tavily Search → Latest news

Example Queries:

- "What is the weather in Bhopal?"
- "Latest news of Delhi"
- "Weather and news of Manali"



Key Concepts Summary

Concept	What It Is	Why It Matters
Runnable	Unified abstraction for all components	Same methods, same SDK
Sequence	Connect components one after another	Basic pipeline
Parallel	Run multiple pipelines simultaneously	Efficiency
Lambda	Custom Python function in pipeline	Flexibility
Pass-Through	Preserve original input	Keep data alongside outputs
Tool	Function + External Service	LLM can interact with the world
Built-in Tool	Pre-built by LangChain	Ready to use
Custom Tool	Created by you	Specific to your needs
Tool Binding	Give tools to LLM	LLM knows what's available
Tool Calling	LLM suggests a tool	LLM decides what to use
Tool Execution	You run the tool	Actual action happens
Human-in-the-Loop	Manual approval before execution	Control & safety
Agent	LLM + Tools + Control Logic	Autonomous decision-making

Concept	What It Is	Why It Matters
create_agent	High-level abstraction	Automates agent creation
Middleware	Add logic between steps	Customize behavior
wrap_tool_call	Middleware for tool calls	Human approval

 Code Examples

Complete City Agent (Manual)

```
python

from dotenv import load_dotenv
load_dotenv()

import os
import requests
from langchain_mistralai import ChatMistralAI
from langchain.tools import tool
from langchain_core.messages import HumanMessage, ToolMessage
from tavily import TavilyClient

# --- Tools ---
@tool
def get_weather(city: str) -> str:
    """Get current weather of a city."""
    api_key = os.getenv("OPENWEATHER_API_KEY")
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}&units=metric"
    response = requests.get(url)
    data = response.json()
    return f"Weather in {city}: {data['weather'][0]['description']}, {data['main']['temp']}"

@tool
def get_news(city: str) -> str:
    """Get latest news about a city."""
    client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))
    response = client.search(query=f"latest news in {city}", max_results=3)
    return "\n\n".join([f"{r['title']}\n{r['url']}" for r in response['results']])
```

```

# --- LLM ---
llm = ChatMistralAI(model="mistral-small-2506")
llm_with_tools = llm.bind_tools([get_weather, get_news])

tools = {"get_weather": get_weather, "get_news": get_news}
messages = []

# --- Agent Loop ---
print("City Intelligence System")
print("Type 'exit' to quit.")

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break

    messages.append(HumanMessage(content=user_input))

    while True:
        result = llm_with_tools.invoke(messages)
        messages.append(result)

        if not result.tool_calls:
            print("Final Answer:", result.content)
            break

        for tool_call in result.tool_calls:
            tool_name = tool_call["name"]
            confirm = input(f"Agent wants to call {tool_name}. Approve? (yes/no): ")

            if confirm.lower() != "yes":
                print("Tool call denied.")
                break

            tool_result = tools[tool_name].invoke(tool_call["args"])
            messages.append(ToolMessage(
                content=str(tool_result),
                tool_call_id=tool_call["id"]
            ))

        continue

```

Complete City Agent (Automated)

python

```

from dotenv import load_dotenv
load_dotenv()

import os
import requests
from langchain_mistralai import ChatMistralAI
from langchain.tools import tool
from langchain.agents import create_agent
from tavily import TavilyClient

# --- Tools ---
@tool
def get_weather(city: str) -> str:
    """Get current weather of a city."""
    api_key = os.getenv("OPENWEATHER_API_KEY")
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}&units=metric"
    response = requests.get(url)
    data = response.json()
    return f"Weather in {city}: {data['weather'][0]['description']}, {data['main']['temp']}"

@tool
def get_news(city: str) -> str:
    """Get latest news about a city."""
    client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))
    response = client.search(query=f"latest news in {city}", max_results=3)
    return "\n\n".join([f"{r['title']}\n{r['url']}" for r in response["results"]])

# --- LLM ---
llm = ChatMistralAI(model="mistral-small-2506")

# --- Agent ---
agent = create_agent(
    model=llm,
    tools=[get_weather, get_news],
    system_prompt="You are a helpful city assistant."
)

# --- Chat Loop ---
print("City Agent | Type 'exit' to quit")
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break

```

```
result = agent.invoke({  
    "messages": [{"role": "user", "content": user_input}]  
})  
  
print("Bot:", result["messages"][-1].content)
```



Requirements File

text

```
langchain  
langchain-core  
langchain-community  
langchain-mistralai  
langchain-openai  
langchain-tavily  
tavily-python  
python-dotenv  
requests  
streamlit  
rich
```



Environment Variables

env

```
MISTRAL_API_KEY=your_key_here  
OPENAI_API_KEY=your_key_here  
TAVILY_API_KEY=your_key_here  
OPENWEATHER_API_KEY=your_key_here
```



What's Next?

Video 4: Big Project with Deployment

- Full project with deployment
- Scalable models

- Production-level RAG + Agentic AI
- Possible project: Perplexity-like app with multiple models

Future Topics:

- **LangGraph** — Low-level orchestration for agents
- **Agentic AI** — Multi-agent systems
- **Deployment** — Making models scalable



Key Takeaways

1. **Runnables** replaced Chains — everything is a Runnable.
2. **LEGO analogy** — All components connect the same way.
3. **Same methods** — `.invoke()` , `.batch()` , `.stream()` .
4. **Tools extend LLM capabilities** — brain + hands.
5. **Tool Calling has 4 steps** — Create, Bind, Call, Execute.
6. **Human-in-the-Loop** — Manual approval before execution.
7. **Agents = LLM + Tools + Control Logic.**
8. `create_agent` — Automates the agent loop.
9. **Middleware** — Add custom logic (e.g., human approval).
10. **Manual vs. Automated** — Manual for learning, automated for production.



Resources

- **LangChain Docs:** python.langchain.com
- **Tavily:** tavily.com
- **OpenWeather:** openweathermap.org
- **Mistral AI:** mistral.ai
- **LangGraph:** langchain-ai.github.io/langgraph

🎯 Conclusion

In this video, we:

1. Learned **Runnables** — Sequence, Parallel, Lambda, Pass-Through.
2. Learned **Tools** — Built-in & Custom.
3. Learned **Tool Calling** — Binding, Calling, Execution.
4. Built **Manual Agent** with Human-in-the-Loop.
5. Built **Automated Agent** with `create_agent`.
6. Built **City Intelligence Agent** project.

Next: Big Project with Deployment.

"You made that decision today. Trust yourself because you are more capable than you will ever know." — Aakash Vyas

End of Notes 📝

give me interview questions with answers topic wise of all the things covered in this chat

🎯 Generative AI Interview Questions & Answers

Topic-Wise (Based on Videos 1, 2 & 3)

📁 TOPIC 1: LLM Foundations

Q1. What is an LLM? Break down the term.

Answer:

LLM = Large Language Model

- **Large** → Trained on huge amounts of text data (books, websites, Wikipedia, code, conversations). Requires massive GPU power and investment.

- **Language** → Understands and generates human language (English, Hindi, Spanish, even programming languages like Python).
- **Model** → A Deep Learning system (Neural Network) built on Transformer architecture that learns patterns from data.

Key Point: LLMs don't "think" or "understand" — they convert text to numbers (embeddings), find patterns, and predict the next most likely word repeatedly until the answer is complete.

Q2. What is the full form of GPT? What architecture is it based on?

Answer:

- **GPT = Generative Pre-trained Transformers**
- Based on **Transformer architecture**
- Introduced in the research paper "**Attention Is All You Need**" by Google Brain
- The entire GenAI ecosystem is built on Transformers

Q3. How does an LLM actually work internally? (Step by step)

Answer:

1. **Input text** → User gives a prompt
2. **Tokenization** → Text is split into tokens
3. **Embeddings** → Tokens converted into numbers (vectors)
4. **Pattern recognition** → Model looks at patterns in embeddings
5. **Prediction** → Predicts the next most likely word
6. **Repeat** → Keeps predicting word-by-word until answer is complete
7. **Output** → Final generated text

Key: LLMs don't search Google, don't think like humans, and don't understand meaning. They only work with numbers and patterns.

Q4. What are Embeddings? Why are they important?

Answer:

Embeddings = Vector representation of data (text → numbers)

- Convert text into numerical form (e.g., 0.6, 0.2, 1.9...)
- Each embedding has a fixed number of dimensions (e.g., 512, 768, 1536, 3072)
- Capture **semantic meaning** — similar texts have similar embeddings

Why important:

- LLMs only understand numbers
- Used in **search, RAG, recommendation systems**
- Help find similar content
- Bridge between human language and machine processing

Example: "Gradient Descent Optimization" → [0.2, 0.6, 1.9, ...] (512 dimensions)

Q5. What is the difference between Human-Friendly and Machine-Friendly output?

Answer:

Human-Friendly	Machine-Friendly (JSON)
"The movie Interstellar was released in 2014 and directed by Christopher Nolan."	<pre>{"title": "Interstellar", "year": 2014, "director": "Christopher Nolan"}</pre>
Easy for humans to read	Easy for machines to parse
Used in chat interfaces	Used in databases, APIs, backend

Machine-friendly (structured output) is crucial for real-world applications because it can be stored, searched, filtered, and analyzed.

Q6. What are the 4 phases of Generative AI evolution?

Answer:

Phase	What Happens
1. Research	Scientists publish papers (LSTM, GRU, "Attention Is All You Need")
2. Base Models	Big companies (OpenAI, Google, Anthropic) create LLMs
3. Providers	Companies provide APIs to use their models
4. You (Developer)	You use these LLMs to build AI applications

Q7. What is the context window? Why is it important?**Answer:**

Context Window = Maximum number of tokens an LLM can process at once.

- Every LLM has a limit (e.g., 32,000 tokens for Mistral Small, 128,000 for GPT-4)
- If you send more than the limit → Error (token limit exceeded)
- **Why important:** Determines how much text you can feed at once
- **Solution for large documents:** Chunking (Text Splitting)

**TOPIC 2: Prompts & Prompt Engineering****Q8. What is a Prompt? What are the 4 types of prompts?****Answer:**

Prompt = Instruction given to a language model.

Without prompts, the model has no direction.

4 Types:

Type	Description	Example
1. Simple Prompt	Direct instruction	"Explain Machine Learning"
2. System + User Prompt	Roles (System sets behavior, User asks)	System: "You are a teacher." User: "Explain LLM."
3. Prompt Template	Reusable prompt with placeholders	"Explain {topic} in simple terms"
4. Structured Prompt	Fixed format (JSON, bullet points)	"Return in JSON format with keys: title, year"

Q9. What is a Prompt Template? Why use it?

Answer:

Prompt Template = Reusable prompt with placeholders.

Why use it:

- **Reusability** — Write once, use many times
- **Consistency** — Same format every time
- **Efficiency** — No need to rewrite prompts
- **Variables** — Replace placeholders dynamically

Example:

```
python
```

```
template = PromptTemplate.from_template(  
    "Explain {topic} in simple terms"  
)  
final_prompt = template.invoke({"topic": "Machine Learning"})
```

Real-world use case: Movie information extraction (Sinés Analytics) — same template, different paragraphs.

Q10. What is the difference between System Message and Human Message?

Answer:

System Message	Human Message
Sets AI's behavior, tone, personality	Represents user input
Given at the beginning	Given every time user asks
Example: "You are a funny AI agent."	Example: "Tell me a joke."
Not shown to user directly	Directly from user

Example:

- System: "You are a sad AI agent. Reply in a sad way."
- Human: "Hello"
- AI: "Hello... I'm here but I wish I wasn't."

Q11. What is Structured Output? Why is it important?

Answer:

Structured Output = AI responses follow a fixed, machine-readable format (like JSON).

Why important:

- Companies need **data**, not text
- Can **store, search, filter, recommend, analyze**
- Can be sent to **APIs**
- Example: Store movie data in MongoDB/SQL

Example:

```
json

{
  "title": "3 Idiots",
  "release_year": 2009,
```

```
"genre": ["Comedy", "Drama"],  
"director": "Rajkumar Hirani",  
"cast": ["Aamir Khan", "R. Madhavan", "Sharman Joshi"],  
"rating": 8.4  
}
```

Q12. What is Pydantic? How is it used for Structured Output?

Answer:

Pydantic = Python library for data validation and schema creation.

How it works:

1. **Create a Schema (Class)** — Defines the structure
2. **Create a Parser** — `PydanticOutputParser`
3. **Add format instructions to prompt**
4. **Parse the response** — Ensures AI output follows the schema

Example:

python

```
from pydantic import BaseModel  
from typing import List, Optional  
  
class Movie(BaseModel):  
    title: str  
    release_year: Optional[int] = None  
    genre: List[str]  
    director: Optional[str] = None  
    cast: List[str]  
    rating: Optional[float] = None  
    summary: str  
  
parser = PydanticOutputParser(pydantic_object=Movie)
```

Why? Ensures AI's output follows the **exact schema** — like a teacher checking a student's form.

Q13. What is Temperature? How does it affect output?

Answer:

Temperature = Controls randomness/creativity of output (0 to 1).

Temperature	Behavior	Use Case
0	Deterministic, logical, same output every time	Math, logic, reasoning tasks
0.5	Balanced	General use
0.7–0.9	Creative, diverse, different outputs	Poems, stories, jokes
1.0	Maximum creativity	Brainstorming

Example:

- Temp 0 → Same poem every time
- Temp 0.9 → Different, advanced poem every time

Q14. What is Max Tokens? Why limit it?

Answer:

Max Tokens = Maximum number of tokens the LLM can generate in response.

Why limit:

- **Cost control** — More tokens = More cost
- **Prevent runaway responses** — Avoid overly long answers
- **Efficiency** — Faster responses

Example: `max_tokens=20` → Only ~20 words generated.

Note: Tokens can be words, characters, or subwords depending on the model.

TOPIC 3: LangChain & Models

Q15. What is LangChain? Why is it needed?

Answer:

LangChain = Open-source framework with pre-built agent architecture.

Problem it solves:

- Each LLM provider (OpenAI, Anthropic, Google) has **different SDKs**
- Switching providers meant **rewriting code**

Solution:

- **Unified Interface** — Same code for any LLM
- Just change the model name, everything else stays the same
- Connects to OpenAI, Anthropic, Google, and more
- Provides: Models, Prompts, Chains/Runnables, Memory, Indexes, Agents

Q16. What are the 6 main components of LangChain?

Answer:

Component	Purpose
1. Models	The LLMs themselves (chat, embedding, multimodal)
2. Prompts	Instructions given to the model
3. Chains/Runnables	Sequence of steps connected together
4. Memory	Remember past interactions
5. Indexes	Connect external data (PDFs, databases) — RAG
6. Agents	AI systems that decide actions dynamically using tools

Q17. What are the 3 main types of models in practical GenAI?

Answer:

Type	Job	Example
1. Chat Models	Generate text, answer questions, summarize	ChatGPT, Gemini, Claude
2. Embedding Models	Convert text → numbers (vectors)	OpenAI Embeddings, HuggingFace
3. Multimodal Models	Work with text + images + audio + video	GPT-4 Vision, Gemini

Q18. What is the difference between `init_chat_model` and provider-specific classes?

Answer:

<code>init_chat_model</code>	Provider-Specific Classes
Generic function	Specific to each provider
<code>init_chat_model("gpt-4.1")</code>	<code>ChatOpenAI(model="gpt-4.1")</code>
<code>init_chat_model("gemini-2.5-flash")</code>	<code>ChatGoogleGenerativeAI(model="gemini-2.5-flash")</code>
Easy to switch providers	More control over configuration
Less code	More explicit

Both work the same way — use `.invoke()` to get responses.

Q19. How do you use Hugging Face models locally vs. via API?

Answer:

Via API (Server-based):

```
python
```

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint

llm = HuggingFaceEndpoint(
    repo_id="deepseek-ai/DeepSeek-R1",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)
```

- Uses Hugging Face servers
- Limited free usage → then paid
- No local storage needed

Locally (Download the model):

```
python
```

```
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline

llm = HuggingFacePipeline.from_model_id(
    model_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation"
)
model = ChatHuggingFace(llm=llm)
```

- Downloads model to your system
- **Free unlimited usage**
- Needs storage (small models: ~2GB, large: ~700GB)

TOPIC 4: RAG (Retrieval Augmented Generation)

Q20. What is RAG? Why is it needed?

Answer:

RAG = Retrieval Augmented Generation

Problem: LLMs don't know:

- Your company's private data
- Your personal documents (PDFs, notes)
- Real-time information
- Specific domain knowledge

Solution: Connect external data sources to LLM → Create a pipeline → Get context-aware answers.

Flow:

text

PDF → Document Loader → Text Splitter → Embeddings → Vector Database

↓

User Query → Embedding → Similarity Search → Retrieve Chunks → LLM → Answer

Why needed:

- LLM only knows what it was trained on
- RAG gives it **external knowledge** without retraining

Q21. What is a Document Loader? Name some types.

Answer:

Document Loader = Converts raw files into Document objects.

Each Document has:

- **page_content** — The actual text
- **metadata** — Source, page number, author, etc.

Types:

Loader	For
TextLoader	.txt files
PyPDFLoader	PDF files
WebBaseLoader	Websites

Loader	For
CSVLoader	CSV files
UnstructuredLoader	Various formats
Cloud (S3, Azure)	Cloud storage
Social (Twitter, Reddit)	Social media

Key Point: A PDF with 15 pages → 15 Document objects (one per page).

Q22. What is Text Splitting (Chunking)? Why is it needed?

Answer:

Text Splitting = Breaking large documents into smaller chunks.

Why needed:

1. **Context Window Limit** — LLMs have max token limits
2. **Better Retrieval** — Smaller chunks = more precise embeddings
3. **More Accurate Embeddings** — One clear idea per chunk
4. **Faster Processing** — Smaller chunks = faster
5. **Faster Similarity Search** — Vector DB search is faster

Example: 534-page PDF → 534 documents → Can't feed all at once → Split into chunks.

Q23. What are the 3 types of Text Splitters?

Answer:

Splitter	How It Works	Best For
1. CharacterTextSplitter	Splits by number of characters	Simple, not recommended

Splitter	How It Works	Best For
2. TokenTextSplitter	Splits by number of tokens (using tiktoken)	Aligns with LLM processing
3. RecursiveCharacterTextSplitter	Tries <code>\n\n</code> → <code>\n</code> → space → any char	Best for RAG

RecursiveCharacterTextSplitter is the most recommended because it:

- Preserves meaning and structure
- Tries to split on paragraphs first
- Falls back to lines, then spaces
- Finally splits on any character

Example:

```
python

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(docs)
```

Q24. What is Chunk Overlap? Why use it?

Answer:

Chunk Overlap = Number of characters/tokens shared between consecutive chunks.

Why use it:

- Preserves meaning across boundaries
- Prevents information loss at chunk edges
- Helps maintain context between chunks

Example:

- Chunk 1: "Gradient descent is an optimization algorithm..."

- Chunk 2: "...optimization algorithm used in machine learning..."
- The overlap ensures the meaning isn't cut off.

Typical values: chunk_size=1000, chunk_overlap=200

Q25. What is a Vector Database? How is it different from a normal database?

Answer:

Normal Database (MySQL, PostgreSQL)	Vector Database (Chroma, Pinecone)
Stores structured data (text, numbers, rows)	Stores embeddings (vectors)
Search by exact match or indexed column	Search by similarity (cosine, Euclidean)
Uses B-tree, hash indexing	Uses ANN algorithms (HNSW, IVF, PQ)
Example: <code>WHERE movie = 'Interstellar'</code>	Example: "Movies about space travel"
Use case: Banking, user records	Use case: AI search, recommendations, RAG

Why Vector DB?

- Normal DB can't handle 1 lakh embeddings × 512 dimensions efficiently
- Vector DB uses **Approximate Nearest Neighbor (ANN)** for fast search
- **IVF example:** Divides 1 lakh embeddings into 5 clusters → Search only in relevant cluster → 5x faster

Q26. What is Similarity Search? How does it work?

Answer:

Similarity Search = Finding the most similar vectors to a query vector.

How it works:

1. Query → Convert to embedding (query vector)

2. Compare query vector with all document vectors
3. Use **cosine similarity**, **Euclidean distance**, etc.
4. Return **top K most similar** documents

Cosine Similarity:

- Measures angle between two vectors
- Smaller angle = more similar
- Most common metric

Euclidean Distance:

- Straight-line distance between vectors
- Smaller distance = more similar

Example: Query "What is gradient descent?" → Finds chunks about gradient descent.

Q27. What is MMR (Maximum Marginal Relevance)?

Answer:

MMR = Balances relevance and diversity in retrieved documents.

Problem with Similarity Search:

- Returns **redundant** results (similar chunks saying the same thing)
- Wastes context window & tokens
- Lacks diversity

How MMR works:

1. Retrieve top **fetch_k** documents (e.g., 10) using similarity
2. Apply MMR to select **k** documents (e.g., 4) that are diverse
3. **Lambda** parameter (0 to 1):
 - $\text{lambda_mult}=1$ → Only relevance (no diversity)
 - $\text{lambda_mult}=0$ → Maximum diversity
 - $\text{lambda_mult}=0.5$ → Balanced

Example:

```
python
```

```
retriever = vector_store.as_retriever(  
    search_type="mmr",  
    search_kwargs={"k": 4, "fetch_k": 10, "lambda_mult": 0.5}  
)
```

Q28. What is Multi-Query Retrieval?

Answer:

Multi-Query Retrieval = Generates multiple variations of a query using LLM, searches for each, combines results.

Problem: A single query might not capture all aspects of a concept.

How it works:

1. User query: "What is gradient descent?"
2. LLM generates:
 - "Explain gradient descent."
 - "How does gradient descent work?"
 - "Optimization algorithm used in neural networks."
3. Retriever searches for each query.
4. Combines all results.

Example:

```
python
```

```
from langchain_classic.retrievers.multi_query import MultiQueryRetriever  
  
multi_query_retriever = MultiQueryRetriever.from_llm(  
    retriever=base_retriever,  
    llm=llm  
)
```



TOPIC 5: Runnables

Q29. What are Runnables? Why were they introduced?

Answer:

Runnables = A unified abstraction where every component is treated the same way.

Problem before Runnables:

- Each component had different functions:
 - `prompt.format_messages()`
 - `model.predict_messages()`
 - `parser.parse()`
- **50+ types of chains** — confusing which to use
- Not flexible — couldn't add custom logic between steps
- No parallel execution

Solution: Everything becomes a Runnable.

- Prompt Template → Runnable
- Model → Runnable
- Output Parser → Runnable
- Retriever → Runnable
- Even your own Python function → Runnable

All Runnables expose the same methods:

- `.invoke()` — Single input → Output
- `.batch()` — Multiple inputs → Multiple outputs
- `.stream()` — Token-by-token output

Analogy: Think of components as **LEGO blocks**. Each block has a different purpose, but they all connect the same way.

Q30. What is a Sequence Runnable?

Answer:

Sequence Runnable = Connects components one after another.

Output of Component 1 → Input of Component 2 → Output of Component 3

Example:

python

```
chain = prompt | model | parser
result = chain.invoke({"topic": "Machine Learning"})
```

Flow:

1. User gives topic → Prompt creates formatted prompt
2. Formatted prompt → Model generates response
3. Response → Parser cleans it up
4. Final result saved

Key: No manual calling — everything is connected automatically.

Q31. What is a Parallel Runnable?

Answer:

Parallel Runnable = Runs multiple pipelines simultaneously on the same input.

When to use:

- You want multiple outputs at the same time
- Example: Short explanation + Detailed explanation
- Example: Same answer in different formats

Example:

python

```
from langchain_core.runnables import RunnableParallel

chain = RunnableParallel(
    short=short_chain,
    detailed=detailed_chain
)

result = chain.invoke({"topic": "Machine Learning"})
```

```
print(result["short"])
print(result["detailed"])
```

Result: Both pipelines run **at the same time**. Result is a dictionary.

Q32. What is Runnable Lambda?

Answer:

Runnable Lambda = Inserts a custom Python function inside a pipeline.

When to use:

- Modify data between steps
- Extract a specific part of a dictionary
- Add custom logic

Example:

```
python

from langchain_core.runnables import RunnableLambda

chain = RunnableParallel(
    short=short_chain,
    detailed=detailed_chain
) | RunnableLambda(lambda x: x["short"])

result = chain.invoke({"topic": "Machine Learning"})
```

How it works:

- Takes a function
- Function receives the input
- Whatever the function returns becomes the output

Q33. What is Runnable Pass-Through?

Answer:

Runnable Pass-Through = Passes the input unchanged to the output.

Example:

python

```
from langchain_core.runnables import RunnablePassthrough

chain = RunnablePassthrough()
result = chain.invoke("Hello")
print(result) # Output: "Hello"
```

When to use:

- Preserve original input
- Use it alongside other outputs in later steps
- Example: You want both the code AND its explanation

Real-world Example: Code Generator + Explainer

python

```
chain = RunnableParallel(
    code=RunnablePassthrough(), # Keep original code
    explanation=explain_prompt | model | parser # Explain it
)
```



TOPIC 6: Tools

Q34. Why do LLMs need Tools?

Answer:

LLMs are powerful but have limitations:

Limitation**Example**

No real-time data

Doesn't know today's news

Limitation	Example
No accurate calculations	Struggles with complex math
No API access	Can't call external APIs
No personal data	Doesn't know your company's data
No actions	Can't send emails, book tickets

Solution: Tools

LLM = Brain. Tools = Hands & Legs.

Without tools, an LLM is like a brain without a body — it can think but can't **do** anything.

Q35. What are Built-in Tools vs Custom Tools?

Answer:

Built-in Tools	Custom Tools
Pre-built by LangChain	Created by you
Ready to use	Based on your needs
Search (Tavily, Google, DuckDuckGo)	Access your database
Code Interpreters	Call your own API
Productivity (Gmail, Slack)	Process your data
Example: <code>TavilySearchResults()</code>	Example: <code>get_greeting()</code>

Built-in Example:

```
python
```

```
from langchain_community.tools.tavily_search import TavilySearchResults
search_tool = TavilySearchResults(max_results=5)
```

Custom Example:

python

```
from langchain.tools import tool

@tool
def get_greeting(name: str) -> str:
    """Generate a greeting message for a user."""
    return f"Hello {name}, welcome to the AI world!"
```

Q36. How do you create a Custom Tool?

Answer:

Step 1: Create a Python function

python

```
def get_greeting(name: str) -> str:
    """Generate a greeting message for a user."""
    return f"Hello {name}, welcome to the AI world!"
```

Step 2: Add @tool decorator

python

```
from langchain.tools import tool

@tool
def get_greeting(name: str) -> str:
    """Generate a greeting message for a user."""
    return f"Hello {name}, welcome to the AI world!"
```

Step 3: Use the tool

python

```
result = get_greeting.invoke({"name": "Aakash"})
print(result) # Output: Hello Aakash, welcome to the AI world!
```

Key Components:

Component	Purpose
Function	The actual logic
Type Hints	Input/output data types
Docstring	Tells the LLM what the tool does
@tool decorator	Converts function into a Tool

Tool Properties:

python

```
print(get_greeting.name)          # "get_greeting"  
print(get_greeting.description)   # "Generate a greeting message for a user."  
print(get_greeting.args)         # {"name": {"title": "Name", "type": "string"}}
```

Q37. What is Tool Binding? Why is it needed?

Answer:

Tool Binding = Giving tools to the LLM so it knows what's available.

Why needed:

- LLM needs to know:
 - What tools are available
 - What they do
 - When to use them

Example:

python

```
llm = ChatMistralAI(model="mistral-small-2506")  
llm_with_tools = llm.bind_tools([get_weather, get_news])
```

Important: When you bind tools, the **entire tool metadata** (name, description, args) is sent to the LLM. This increases **input tokens**.

Example: Without tools → 5 input tokens. With tools → 68 input tokens.

Q38. What is Tool Calling? Explain the 4 steps.

Answer:

Tool Calling = LLM suggests which tool to use, but does NOT execute it.

4 Steps:

Step	What Happens
1. Create Tool	@tool decorator on function
2. Bind Tool	llm.bind_tools([tool])
3. Tool Calling	LLM suggests which tool to use (result.tool_calls)
4. Tool Execution	You execute the tool and get the result

Example:

python

```
# Step 1: Create
@tool
def get_text_length(text: str) -> int:
    """Returns the number of characters in a given text."""
    return len(text)

# Step 2: Bind
llm_with_tools = llm.bind_tools([get_text_length])

# Step 3: Call (LLM suggests)
result = llm_with_tools.invoke("Return the number of characters in 'Hello how are you?')
print(result.tool_calls)
# [{"name": "get_text_length", "args": {"text": "Hello how are you?"}, "id": "call_abc123"}]

# Step 4: Execute (You run it)
tool_result = get_text_length.invoke(result.tool_calls[0]["args"])
print(tool_result) # Output: 17
```

Q39. What is Human-in-the-Loop in Tool Calling?

Answer:

Human-in-the-Loop = Manual approval before executing a tool.

Why?

- You want to **approve** before executing a tool
- Example: Before booking a ticket, you want confirmation

How it works:

1. LLM suggests tool call
2. You ask the user: "Agent wants to call `get_weather` . Approve? (yes/no)"
3. If yes → Execute the tool
4. If no → Deny and continue

Example:

python

```
if result.tool_calls:
    tool_call = result.tool_calls[0]
    tool_name = tool_call["name"]

    confirm = input(f"Agent wants to call {tool_name}. Approve? (yes/no): ")

    if confirm.lower() == "yes":
        tool_result = tools[tool_name].invoke(tool_call["args"])
        tool_message = ToolMessage(
            content=str(tool_result),
            tool_call_id=tool_call["id"]
        )
        messages.append(tool_message)
    else:
        print("Tool call denied by user.")
```

Message History:

- **Human Message** → User's query
- **AI Message** → LLM's tool suggestion
- **Tool Message** → Tool's result

- **AI Message** → Final answer

TOPIC 7: Agents

Q40. What is an Agent?

Answer:

Agent = LLM (Brain) + Tools (Hands) + Control Logic (Code)

Agent vs. Human-in-the-Loop:

- **Human-in-the-Loop:** You manually execute tools
- **Agent:** Tools are executed **automatically** in a loop

Example: Planning an Evening

- Check weather → Book cab → Order food → Book movie tickets
- Multiple decisions, multiple tools, multiple steps
- An agent does all of this **autonomously**

How it works:

1. User gives a query
2. LLM decides which tool(s) to use
3. Tools are executed automatically
4. Results are fed back to LLM
5. LLM generates final answer

Q41. What is the difference between Manual Agent and Automated Agent?

Answer:

Feature	Manual Agent	Automated Agent (<code>create_agent</code>)
Code	You write the loop	LangChain handles it
Control	Full control over every step	Less control, more automation
Visibility	Every step visible	Hidden behind the scenes
Boilerplate	Lots of repetitive code	Minimal code
Error-prone	Yes (manual loops)	No (handled internally)
Scaling	Hard to scale	Easy to scale
Best for	Learning, debugging	Production, real projects

Manual Agent:

python

```
while True:
    result = llm_with_tools.invoke(messages)
    messages.append(result)

    if not result.tool_calls:
        print(result.content)
        break

    for tool_call in result.tool_calls:
        tool_result = tools[tool_call["name"]].invoke(tool_call["args"])
        messages.append(ToolMessage(
            content=str(tool_result),
            tool_call_id=tool_call["id"]
        ))
```

Automated Agent:

python

```
from langchain.agents import create_agent

agent = create_agent(
    model=llm,
    tools=[get_weather, get_news],
    system_prompt="You are a helpful city assistant."
```

```
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "What is the weather in Bhopal?"}]
})
```

Q42. What is create_agent ? How does it work?

Answer:

`create_agent` = High-level abstraction provided by LangChain that automates agent creation.

What it automates:

- The loop
- Tool calls
- Message management
- Stop conditions

How to use:

```
python

from langchain.agents import create_agent

agent = create_agent(
    model=llm,                # The LLM (brain)
    tools=[get_weather, get_news], # The tools (hands)
    system_prompt="You are a helpful city assistant."
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "What is the weather in Bhopal?"}]
})

print(result["messages"][-1].content)
```

Key Point: You just define the model and tools. The framework takes care of the entire execution process internally.

Q43. What is Middleware in `create_agent` ? What is `wrap_tool_call` ?

Answer:

Middleware = Something you add in between an automated process.

`wrap_tool_call` = **Middleware for tool calls (e.g., human approval).**

Example: Human-in-the-Loop with `create_agent`

python

```
from langchain.agents.middleware import wrap_tool_call

@wrap_tool_call
def human_approval(request, handler):
    """Ask for human approval before every tool call."""
    tool_name = request.tool_call["name"]

    confirm = input(f"Agent wants to call {tool_name}. Approve? (yes/no): ")

    if confirm.lower() != "yes":
        return ToolMessage(
            content="Tool call denied by user.",
            tool_call_id=request.tool_call["id"]
        )

    return handler(request)

agent = create_agent(
    model=llm,
    tools=[get_weather, get_news],
    middleware=[human_approval]
)
```

What is Middleware?

- Something you add **in between** an automated process
- Here, it adds a **human approval step** before tool execution

Q44. What is the role of `RunnablePassthrough` in an Agent?

Answer:

`RunnablePassthrough` **preserves the original input while other pipelines run.**

Real-world Example: Code Generator + Explainer

Problem: You want an AI that:

1. Generates code
2. Explains the code
3. Returns both

Solution:

```
python

from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# Step 1: Generate code
code_chain = code_prompt | model | parser

# Step 2: Parallel - keep code + explain it
chain = RunnableParallel(
    code=RunnablePassthrough(), # Keep original code
    explanation=explain_prompt | model | parser # Explain it
)

# Full pipeline
full_chain = code_chain | chain

result = full_chain.invoke({"topic": "Write a palindrome code in Python"})
print(result["code"])
print(result["explanation"])
```

Why Pass-Through?

- Without it, you'd lose the original code after explaining it
- With it, you keep both the code AND the explanation

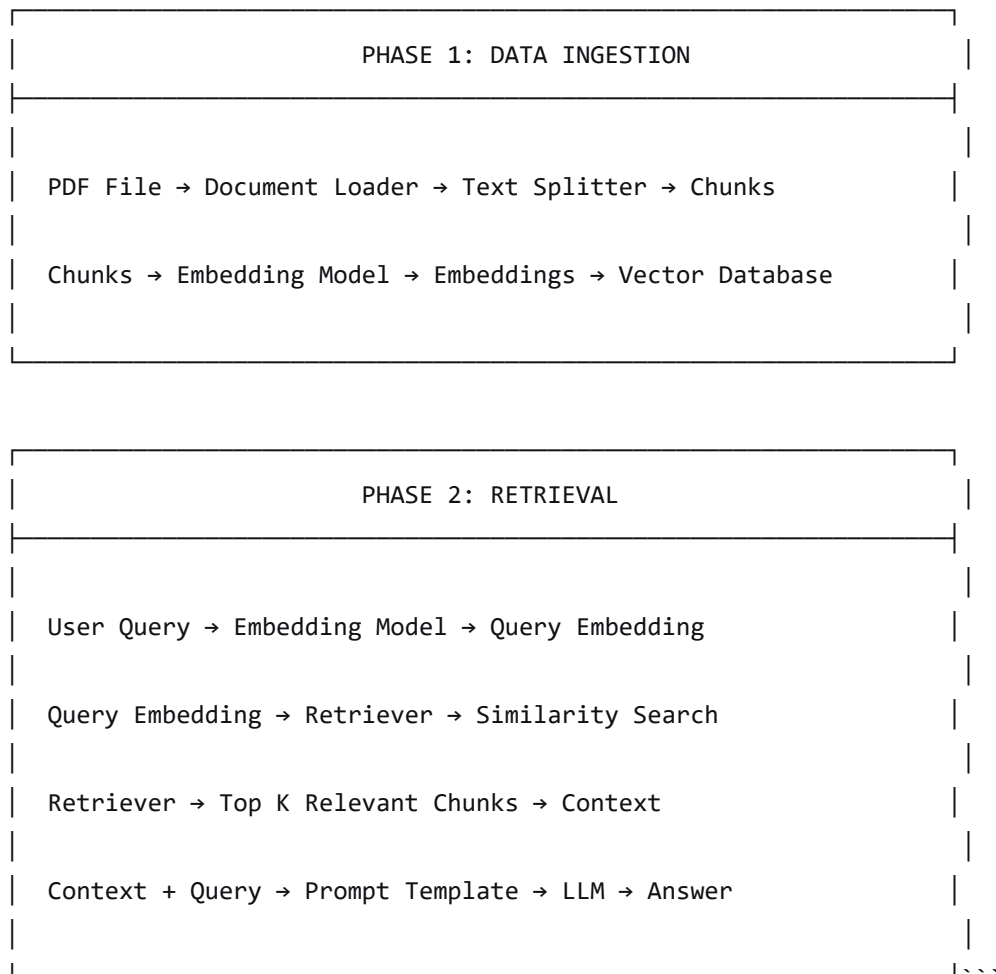


TOPIC 8: Architecture & Design

Q45. Explain the complete RAG pipeline.

Answer:

text



****Steps:****

1. ****Load**** – Document Loader converts PDF to Document objects
2. ****Split**** – Text Splitter creates chunks
3. ****Embed**** – Embedding Model converts chunks to vectors
4. ****Store**** – Vector Database stores embeddings
5. ****Query**** – User asks a question
6. ****Embed Query**** – Query converted to embedding
7. ****Retrieve**** – Similarity search finds relevant chunks
8. ****Context**** – Top chunks form the context
9. ****Generate**** – LLM generates answer from context + query

Q46. Explain the complete Agent pipeline.

****Answer:****

User Query → LLM → Decides which tool(s) to use

↓

Tool(s) Executed

↓

Results fed back to LLM

↓

LLM generates final answer

↓

If more tools needed → Loop back

↓

Final Answer

text

****Detailed Flow:****

1. ****User Input**** → Human Message added to history
2. ****LLM Invoke**** → LLM decides if tool is needed
3. ****Tool Call**** → LLM suggests tool(s)
4. ****Tool Execution**** → Tool(s) run, results added to history
5. ****Loop**** → Repeat until no more tool calls
6. ****Final Answer**** → LLM generates final response

****Key Components:****

- ****LLM (Brain)**** → Decides what to do
- ****Tools (Hands)**** → Perform actions
- ****Control Logic (Code)**** → Manages the loop

🌀 TOPIC 9: Practical & Implementation

Q47. How do you set up a virtual environment for a GenAI project?

****Answer:********Step 1: Install UV (Fast Package Manager)****

```bash

# Mac/Linux

curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)

powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

## Step 2: Create Virtual Environment

```
bash
```

```
uv venv
```

### Step 3: Activate

```
bash
```

```
Mac/Linux
```

```
source .venv/bin/activate
```

```
Windows
```

```
.venv\Scripts\activate
```

### Step 4: Install Packages

```
bash
```

```
uv pip install langchain langchain-community langchain-openai
```

### Why Virtual Environment?

- All dependencies stay inside the folder
- Doesn't mess with your base system
- Always create for any project (GenAI, ML, DL)

### Q48. What is UV? Why use it over pip?

**Answer:**

**UV = Extremely fast Python package & project manager written in Rust.**

**Why UV over pip:**

- **10–100x faster** than pip
- Faster package installation
- Faster dependency resolution
- Written in Rust for performance

**Example:**

```
bash
```

```
uv pip install langchain # Much faster than pip install langchain
```

**Note:** UV is not a replacement for pip — it's an alternative that's much faster.

## Q49. How do you store API keys securely?

**Answer:**

### Step 1: Create .env file

```
env
```

```
OPENAI_API_KEY=your_key_here
MISTRAL_API_KEY=your_key_here
TAVILY_API_KEY=your_key_here
OPENWEATHER_API_KEY=your_key_here
HUGGINGFACEHUB_ACCESS_TOKEN=your_token_here
```

### Step 2: Load in Python

```
python
```

```
from dotenv import load_dotenv
load_dotenv()
```

### Step 3: Add to .gitignore

```
text
```

```
.env
```

**Important:**

- **Never share API keys** with anyone or any AI
- Store only in .env file
- Add .env to .gitignore if using Git

## Q50. What is the difference between `invoke()` , `batch()` , and `stream()` ?

Answer:

| Method                 | Purpose                            | Example                                                             |
|------------------------|------------------------------------|---------------------------------------------------------------------|
| <code>.invoke()</code> | Single input → Single output       | <code>chain.invoke({"topic": "ML"})</code>                          |
| <code>.batch()</code>  | Multiple inputs → Multiple outputs | <code>chain.batch([{"topic": "ML"}, {"topic": "DL"}])</code>        |
| <code>.stream()</code> | Token-by-token output (real-time)  | <pre>for chunk in chain.stream({"topic": "ML"}): print(chunk)</pre> |

All **Runnables** expose these methods — that's the power of the unified interface.

## Q51. How do you handle errors in a RAG pipeline?

Answer:

Common errors:

1. **Context window exceeded** → Use chunking
2. **API rate limit** → Add retry logic
3. **Empty retrieval** → Check embeddings/vector store
4. **Invalid JSON output** → Use Pydantic parser
5. **Tool call errors** → Use `wrap_tool_call` middleware

Example: Error handling in tool execution

```
python

try:
 tool_result = tools[tool_name].invoke(tool_call["args"])
except Exception as e:
 tool_result = f"Error: {str(e)}"
```

Example: Human-in-the-Loop for safety

python

```
confirm = input(f"Approve tool call? (yes/no): ")
if confirm.lower() != "yes":
 return ToolMessage(content="Tool call denied.", tool_call_id=...)
```

## Q52. What is the difference between LangChain and LangGraph?

Answer:

| LangChain                | LangGraph                         |
|--------------------------|-----------------------------------|
| High-level framework     | Low-level orchestration framework |
| create_agent abstraction | Full control over agent workflows |
| Easy to use              | More complex, more powerful       |
| Good for simple agents   | Good for complex, stateful agents |
| Less control             | Full control over every step      |
| Hides complexity         | Exposes complexity                |

When to use:

- **LangChain** → Simple agents, quick prototyping, learning
- **LangGraph** → Production agents, complex workflows, stateful agents

**LangGraph is used by:** Klarna, Uber, JP Morgan, and more.



## TOPIC 10: Projects & Real-World

## Q53. Explain the Sinés Analytics project.

Answer:

**Sinés Analytics = Media intelligence company that organizes and analyzes movie data.**

**Problem:**

- Streaming platforms (Netflix, YouTube, Amazon) need **structured movie metadata**
- Incoming data arrives as **messy, unstructured paragraphs**
- Extracting manually is **time-consuming, expensive, error-prone**

**Solution:**

- Build an AI-powered tool that:
  1. Takes a raw paragraph about a movie
  2. Extracts structured information (title, year, genre, director, cast, etc.)
  3. Generates a clean summary
  4. Returns it in **JSON format**

**Tech Stack:**

- LangChain
- Mistral AI (free model)
- Pydantic (for schema validation)
- Streamlit (for UI)
- Prompt Templates

**Workflow:**

1. User pastes a movie paragraph
2. Prompt Template sends it to LLM with system instructions
3. LLM extracts information
4. Pydantic Parser validates the output
5. Structured JSON is returned

**Q54. Explain the Coursemate AI project.****Answer:**

**Coursemate AI = AI-powered study assistant that helps students interact with their learning materials.**

**Problem:**

- Students rely on multiple sources: lecture notes, textbooks, PDFs, research papers
- These documents are **long and difficult to navigate**
- Finding specific information is **time-consuming**

**Solution:**

- Students can **chat with their study materials**
- Ask questions → Get accurate answers extracted from documents

**Tech Stack:**

- Document Loaders (PyPDFLoader)
- Text Splitters (RecursiveCharacterTextSplitter)
- Embeddings (OpenAI Embeddings / HuggingFace)
- Vector Database (Chroma DB)
- Retriever (MMR Strategy)
- LLM (Mistral AI)
- Streamlit (UI)

**Flow:**

text

PDF → Load → Split → Embed → Store in Chroma DB

↓

User Query → Embed → Similarity Search → Retrieve Chunks → LLM → Answer

**Q55. Explain the City Intelligence Agent project.****Answer:**

**City Intelligence Agent = AI agent that gives real-time weather + news for any city.**

**Tools:**

1. **get\_weather(city)** → OpenWeather API → Real-time weather
2. **get\_news(city)** → Tavily Search → Latest news

LLM: Mistral Small (free)

Approach:

- **Manual Loop** (Human-in-the-Loop) — You approve before tool execution
- **Automated** ( `create_agent` ) — Tools execute automatically

Example Queries:

- "What is the weather in Bhopal?"
- "Latest news of Delhi"
- "Weather and news of Manali"

Flow:

text

User Query → LLM → Decides which tool(s) to use → Executes tools → Gets results → Generates final answer

Q56. What is the difference between RAG and Agents?

Answer:

| RAG                              | Agents                               |
|----------------------------------|--------------------------------------|
| Retrieves relevant information   | Decides which tool to use            |
| <b>Static</b> — Always retrieves | <b>Dynamic</b> — Decides at runtime  |
| Used for Q&A from documents      | Used for tasks & actions             |
| No tool execution                | Executes tools (API calls, DB, etc.) |
| Example: Coursemate AI           | Example: City Intelligence Agent     |
| Simpler                          | More complex                         |
| One-shot retrieval               | Loop-based decision making           |

**RAG Flow:** Query → Retrieve → Generate Answer

**Agent Flow:** Query → Decide Tool → Execute → Loop → Final Answer

### Q57. What is the difference between LangChain Chains and Runnables?

Answer:

| Chains (Old)                                | Runnables (New)                                                                          |
|---------------------------------------------|------------------------------------------------------------------------------------------|
| 50+ types (LLMChain, SequentialChain, etc.) | One unified concept                                                                      |
| Different functions for each component      | Same methods ( <code>.invoke()</code> , <code>.batch()</code> , <code>.stream()</code> ) |
| Not flexible                                | Flexible                                                                                 |
| No parallel execution                       | Parallel execution                                                                       |
| Confusing                                   | Clear                                                                                    |
| Removed in LangChain v2                     | Introduced in LangChain v2                                                               |

**Key Insight:** Think of components as **LEGO blocks**. Each block has a different purpose, but they all **connect the same way**.

### Q58. What is the difference between `init_chat_model` and `bind_tools` ?

Answer:

| <code>init_chat_model</code>            | <code>bind_tools</code>             |
|-----------------------------------------|-------------------------------------|
| Initializes a chat model                | Binds tools to a model              |
| <code>init_chat_model("gpt-4.1")</code> | <code>llm.bind_tools([tool])</code> |
| Returns a model instance                | Returns a model with tools          |

`init_chat_model`

`bind_tools`

Used to create the model

Used to give tools to the model

No tools bound

Tools bound

Example:

```
python

Initialize model
model = init_chat_model("gpt-4.1")

Bind tools
model_with_tools = model.bind_tools([get_weather, get_news])
```

Q59. What is the difference between RunnableParallel and RunnableLambda ?

Answer:

RunnableParallel

RunnableLambda

Runs multiple pipelines simultaneously

Inserts a custom function

Returns a dictionary of results

Returns the function's output

Used for multiple outputs

Used for custom logic

Example: Short + Detailed explanation

Example: Extract `x["short"]`

No custom logic

Custom logic

Example:

```
python

RunnableParallel
chain = RunnableParallel(
 short=short_chain,
 detailed=detailed_chain
```

```
)

RunnableLambda
chain = RunnableLambda(lambda x: x["short"])
```

## Q60. What is the difference between RunnablePassthrough and RunnableLambda ?

### Answer:

| RunnablePassthrough                | RunnableLambda                                  |
|------------------------------------|-------------------------------------------------|
| Passes input unchanged             | Modifies input                                  |
| Returns same input                 | Returns function output                         |
| No custom logic                    | Custom logic                                    |
| Example: Keep original code        | Example: Extract a key                          |
| <code>RunnablePassthrough()</code> | <code>RunnableLambda(lambda x: x["key"])</code> |

### Example:

```
python

Pass-Through: Keep input
chain = RunnableParallel(
 code=RunnablePassthrough(), # Same input
 explanation=explain_chain
)

Lambda: Modify input
chain = RunnableLambda(lambda x: x["short"])
```



## Quick Reference: Key Definitions

| Term              | Definition                                                    |
|-------------------|---------------------------------------------------------------|
| LLM               | Large Language Model — generates text based on patterns       |
| Embedding         | Vector representation of text (numbers)                       |
| RAG               | Retrieval Augmented Generation — connect external data to LLM |
| Chunking          | Splitting large documents into smaller pieces                 |
| Vector DB         | Database for storing and searching embeddings                 |
| Retriever         | Fetches relevant chunks from vector DB                        |
| Runnable          | Unified abstraction for all components                        |
| Tool              | Function + External Service for LLM                           |
| Agent             | LLM + Tools + Control Logic                                   |
| Human-in-the-Loop | Manual approval before execution                              |
| Middleware        | Logic added between automated steps                           |
| Structured Output | Machine-readable format (JSON)                                |
| Pydantic          | Python library for schema validation                          |
| Temperature       | Controls creativity (0 = logical, 1 = creative)               |
| Max Tokens        | Limit on output length                                        |
| Context Window    | Max tokens LLM can process at once                            |

## Final Tips for Interviews

1. **Understand the "Why"** — Don't just memorize definitions. Understand why each component exists.
2. **Know the Flow** — RAG flow, Agent flow, Tool calling flow.

3. **Hands-on Experience** — Build projects. Interviewers love practical knowledge.
4. **Stay Updated** — LangChain updates frequently. Know the latest features ( `create_agent` , `wrap_tool_call` ).
5. **Be Honest** — If you don't know something, say "I haven't worked with that, but I can learn."
6. **Think LEGO** — Runnable = LEGO blocks. Everything connects the same way.
7. **Brain + Hands** — LLM = Brain, Tools = Hands, Agent = Both + Control Logic.
8. **Manual vs Automated** — Know when to use each.
9. **Cost Awareness** — Tokens cost money. Use `max_tokens` , chunking, MMR wisely.
10. **Security** — Never share API keys. Use `.env` files.

*End of Interview Questions & Answers* 🎯